

量子コンピューティング 副読本 ― FTQC 編 第 II 部

(スタビライザ符号と論理量子操作)

量子コンピューティング I 講義チーム

目次

第 6 章	スタビライザ形式 — 動かさない演算子に群を与える	1
6.1	測っていた演算子が、符号を定義していた	1
6.2	独立なスタビライザを一本増やすたび、固定される空間は半分になる	2
6.3	生成子が論理演算子と可換だから、測定は誤りだけを映す	4
6.4	演習	6
第 7 章	CSS 構成と Steane 符号 — 古典符号 2 枚から量子符号へ	7
7.1	片手の符号は、繰り返してなくてよい	7
7.2	シンジロームが、反転したビットの番地を直に指す	8
7.3	検査行列の各行が、そのまま生成子になる	8
7.4	同じ符号を二枚使うと、 X と Z を入れ替えても符号は動かない	10
7.5	演習	11
第 8 章	論理状態と論理演算子	13
8.1	論理 0 は、七つの 0 ではない	13
8.2	論理 0 を取り出すと、八つの番地に散らばる	14
8.3	論理 1 は全ビットを裏返した八項、測れば偶奇で見分けがつく	15
8.4	八項は選んだのではなく、三つの独立な反転に強いられた	16
8.5	論理ビットは、符号空間の上でひとつのパウリ代数を回す	17
8.6	演習	18
第 9 章	transversal ゲートとフォールトトレランス — 単純さと耐性が同じ仕掛け	19
9.1	一様な操作が、論理ゲートになり、しかも誤りを増やさない	19
9.2	一様な H が論理アダマールになるのは、手品ではない	20
9.3	ブロック間の CNOT は、誤りを兄弟に渡さない	21
9.4	全量子ビットを測れば、古典デコードが論理ビットを返す	23
9.5	単純さと耐性は、transversal という一つの構造の両面である	24
9.6	演習	26
第 10 章	符号の比較とスタビライザ符号の限界 — なぜ Clifford だけでは足りないか	29
10.1	Steane を選んだ理由は、並べて初めて見える	29

10.2	距離が同じ三符号は、weight と構造で分かれる	30
10.3	五量子ビットの「完全」は、CSS の対称と引き換えである	31
10.4	軽い生成子は、しきい値を押し上げる	33
10.5	どの符号も、transversal だけでは Clifford の天井を越えない	34
10.6	演習	35
付録 A	記法・記号一覧	37
A.1	体・ビット演算と量子状態	37
A.2	演算子とゲート	38
A.3	スタビライザ符号と符号パラメータ	38
A.4	古典線形符号と GF(2) 線形代数	38
A.5	しきい値 (半定量)	39
付録 B	GF(2) 上の線形代数	41
B.1	二元体 $\mathbb{F}_2$ とベクトル空間	41
B.2	$\mathbb{F}_2$ 上の行列とガウス消去・階数	42
B.3	行空間・核と次元	44
B.4	線形符号と検査行列	44
B.5	双対包含と自己双対	45
付録 C	演習解答	47
C.1	第 6 章	47
C.2	第 7 章	50
C.3	第 8 章	53
C.4	第 9 章	59
C.5	第 10 章	64

第 6 章

スタビライザ形式 —— 動かさない演算子に群を与える

6.1 測っていた演算子が、符号を定義していた

第 3 章の終わりに、約束を一つ残した。Shor 符号で「符号空間を動かさない演算子」—— Z_1Z_2 や Z_2Z_3 のような内側の Z 検査の積 —— は、掛け合わせても動かさない性質を保ち、

$$(Z_1Z_2)(Z_2Z_3) = Z_1Z_3$$

のように一つのまとまりを作る。そのまとまりは、いくつもの符号を横並びに扱い始めるとき名前を与えられ、符号を語る共通の言葉になる、と述べた。本章は、その名前を与える側に立つ。名前は**スタビライザ群** (stabilizer group) である。

名前を与える前に、本編で一度通った折り返しを思い出しておく。#9 で誤りを読み取るとき、私たちはブロックをまたぐ検査 —— $Z_1Z_2 \cdot Z_2Z_3$ のような演算子 —— を測ってシンδροームを得ていた。あのとき測っていた相手は、計算に便利だからと適当に選んだ演算子ではない。それは符号空間そのものを定義する演算子だった。符号語をすべて固有値 $+1$ で素通りさせ、符号語でない状態をそうでないと弾く —— 測定とは、状態がその「 $+1$ で固定される空間」に入っているかを問う操作だったのである。測る相手と、符号を定義する相手は、同じものだった。

ここから二つの問いが立つ。第一に、第 3 章で「一つのまとまり」と呼んだものは、構造として何か。掛け合わせて閉じ、単位元を含み、各演算子が逆元をもつ —— 群の言葉そのものに見える。ならば群なのか、どんな群なのか。第二に、「測ってもデータが壊れない」を、直感ではなく構造で言えるか。第 3 章では「スタビライザは論理的な自由に触れない作りだから」と言葉で述べた。その「触れない」を、可換性という代数の関係に翻訳して証明できるか。

先に行き先を置いておく。一つ目の答えは、そのまとまりが**アーベル群 (可換群)** であり、かつ -1 を含まないという二条件で特徴づけられる、というものだ。可換であることが共通の固有空間を持たせ、 -1 を含まないことがその $+1$ 固有空間を空でなくする。二つ目の答えは、生成子 (本編のジェネレータと同じ) が論理演算子と可換であること —— $[g, L] = 0$ —— に尽きる。可換だから、生成子を測っても論理状態は回らず、測定で得た ± 1 は誤りの居場所だ

けを映す。第3章で名を伏せた性質は、この可換性の言い換えだったのである。

二つの答えを、パウリ演算子の最小限の代数から順に組み上げていく。

6.2 独立なスタビライザを一本増やすたび、固定される空間は半分になる

まず、舞台になる演算子の全体を確定する。一個の量子ビットに作用するパウリ演算子は X, Y, Z と恒等 1 の四つで、これに $\pm 1, \pm i$ の位相をつけたものを考える。位相 $\pm i$ まで含めるのは、積で閉じさせるためだ。たとえば

$$XZ = -iY$$

のように、二つの積はしばしば i の係数を伴う。 n 量子ビットでは、各桁にこの四つのどれかをテンソル積で並べ、全体に位相をつけたものをパウリ演算子と呼び、その全体をパウリ群 $\mathcal{P}_n$ と書く。各演算子の重さ (weight) は、恒等でない桁の数 — 実際に手を触れている量子ビットの数 — と定める。 $Z_1 Z_2$ の重さは 2、 $X_1 X_2 X_3$ の重さは 3 である。

次に、二つのパウリ演算子が「並べ替えられるか」を見る。一個の量子ビットの上では、相異なる二つの非恒等パウリ (たとえば X と Z) は順序を入れ替えると符号が反転し — $XZ = -ZX$ — 同じもの同士や片方が恒等なら符号は変わらない。テンソル積に並んだ二つの演算子 A と B では、桁ごとにこの符号が掛かるので、

$$AB = (-1)^k BA$$

となる。 k は、 A と B が桁の上で反符号になる — 相異なる非恒等パウリが重なっている — 桁の数である。

命題 6.1 (重なり偶数則) パウリ演算子 A, B が可換 $AB = BA$ となるのは、両者が相異なる非恒等パウリをもつ桁 (反符号の重なり) の数が偶数のときに限る。これを GF(2) 表現で書くと、 $A = (x | z), B = (x' | z')$ (x_j は第 j 桁が X か Y のとき 1、 z_j は Z か Y のとき 1) に対し、シンプレクティック内積

$$\langle A, B \rangle = x \cdot z' + z \cdot x' \pmod{2}$$

が 0 のとき可換、1 のとき反可換である。

反可換な桁が偶数個あれば符号 (-1) が偶数回掛かって $+1$ に戻り、全体としては並べ替えられる。一個ずつ見れば衝突していても、衝突の数が偶数なら全体は折り合う、ということだ。

では、符号を定義する演算子の集まりには、どんな条件が要るのか。私たちはこれらを生成子として何本も測り、返ってくる ± 1 の組をシンδροームとして読む。複数を矛盾なく測るには、測定どうしが互いを乱してはならない — すなわち生成子は互いに可換でなければならない。実際、反可換な二本 g, h ($\{g, h\} = 0$) を混ぜると、両方を $+1$ に保つ状態は存在しない。もし $g|\psi\rangle = |\psi\rangle$ かつ $h|\psi\rangle = |\psi\rangle$ なら $gh|\psi\rangle = |\psi\rangle$ だが、 $gh = -hg$ より

$$gh|\psi\rangle = -hg|\psi\rangle = -|\psi\rangle$$

となり、 $|\psi\rangle = -|\psi\rangle$ 、すなわち $|\psi\rangle = 0$ に追い込まれる。可換であって初めて、すべてを同時に対角化でき、すべてが $+1$ になる一つの共通固有空間が定まる —— これを**符号空間**と呼ぶ。だから集めるのは、互いにすべて可換なパウリ演算子だ。これらを生成子とする部分群を

$$S = \langle g_1, \dots, g_s \rangle$$

と書く。これは可換群 (**アーベル群**) にほかならない。さらに一つだけ条件を課す —— $-1 \notin S$ 。もし -1 が S に入っていれば、符号語 $|\psi\rangle$ は $-1|\psi\rangle = +|\psi\rangle$ 、すなわち $-|\psi\rangle = |\psi\rangle$ を満たさねばならず、そんな状態は 0 しかない。可換性が共通の $+1$ 固有空間を生み、 -1 を含まないことがそれを空でなく保つ。

どれだけの空間が残るかを、具体物で測る。第 3 章で扱った Shor の、ブロック 1 の内側だけを取り出す。三個の量子ビットに $S = \langle Z_1 Z_2, Z_2 Z_3 \rangle$ を課したものだ (これは Shor の一部であり、最小の例でもある)。生成子は二本とも Z だけでできているので互いに可換で、 -1 は積からは出てこない。固定される空間を、射影子で取り出してみる。一本のパウリ生成子 g は $g^2 = 1$ をみたし固有値は ± 1 だから、その $+1$ 固有空間への射影子は $(1 + g)/2$ と書ける (射影測定が $+1$ 固有空間への射影であることは準備章・第 1 章で見た)。生成子が複数あって互いに可換なら、射影子はそのまま積になる。

$$\Pi = \frac{1 + Z_1 Z_2}{2} \cdot \frac{1 + Z_2 Z_3}{2}.$$

最初の因子 $(1 + Z_1 Z_2)/2$ は「1 番目と 2 番目が同じ値」の状態 —— $|00\rangle$ と $|11\rangle$ —— に射影する。三量子ビットの八次元から、ちょうど半分の四次元が残る。次の因子 $(1 + Z_2 Z_3)/2$ は「2 番目と 3 番目が同じ値」を要求し、四次元をさらに半分の二次元に絞る。残るのは三つの桁がすべて揃った状態、 $|000\rangle$ と $|111\rangle$ の張る二次元である。生成子を独立に一本増やすたび、固定される空間はきっかり半分になる —— 八 $\rightarrow$ 四 $\rightarrow$ 二、独立な生成子 2 本で 2^3 から 2^1 へ。

そして、この生成子こそが、測る相手である。生成子 $Z_1 Z_2$ を測れば 1 番目と 2 番目のパリティ ($+1$ なら同じ、 -1 なら違う) が返り、 $Z_2 Z_3$ を測れば 2 番目と 3 番目のパリティが返る。二本の測定結果の組 $(\pm 1, \pm 1)$ が**シンドローム**だ。第 3 章で「ブロックに位相反転がある」と教えてくれたあの検査は、いま見ているこの S の生成子を測ることだったのである。符号を定義する生成子と、シンドロームを読む測定対象は、最初から同じ一つの集合だった。

この測定がしていることを、空間の側から見ておく。スタビライザを一本立てるごとに、量子ビットが張る空間は固有値 ± 1 で二つに割れる。 s 本そろえば 2^s 個の部分空間に分かれ、シンドロームの組は、いま状態がどの部分空間にいるかを指す住所になる。そのうち、すべての生成子が $+1$ を返す一つの部分空間 —— それが符号空間であり、符号語が正しく収まっているべき場所だ。どこかの量子ビットに X (ビット反転) が起きると、その桁を見張る生成子が -1 に転んで状態を別の部分空間へ弾き飛ばし、シンドロームの組がその行き先を名指す。訂正とは、弾かれた状態をこの全 $+1$ の部分空間へ連れ戻すことにほかならない。

6.3 生成子が論理演算子と可換だから、測定は誤りだけを映す

観察で見た半減を、一般化する。互いに可換で独立な — どの一本も他の積では書けない — 生成子が s 本あれば、射影子 $\prod_i (1 + g_i)/2$ は 2^n 次元を独立に s 回半分にし、符号空間の次元は 2^{n-s} になる。 $-1 \notin S$ がこの空間を空でなく保つことは観察で見たとおりだ。

定理 6.2 (符号空間の次元) 互いに可換で独立な s 本の生成子からなるスタビライザ群 S ($-1 \notin S$) に対し、符号空間

$$\mathcal{C} = \{ |\psi\rangle : g|\psi\rangle = |\psi\rangle \text{ for all } g \in S \}$$

の次元は

$$\dim \mathcal{C} = 2^{n-s}$$

である。これを 2^k と書けば、 $k = n - s$ が符号の運べる論理量子ビットの数になる。

物理量子ビット数 n 、論理量子ビット数 k 、そして誤りに対する強さを測る距離 d (独立した三つ目の量で、ここでは名前だけ与える) を束ねて、符号を $[[n, k, d]]$ と表す。Shor は 9 個の物理量子ビットに 8 本の生成子で $k = 9 - 8 = 1$ 、すなわち $[[9, 1, 3]]$ 。Steane は $k = 7 - 6 = 1$ で $[[7, 1, 3]]$ 。どちらも論理 1 量子ビットを、距離 3 で守っている。

符号ごとに、この S の中身は違う。観察の三量子ビット片は $\langle Z_1 Z_2, Z_2 Z_3 \rangle$ の二本だった。第3章で実際に扱った 9 量子ビットの Shor 符号を、いまの言葉で書き下しておく。生成子は八本で、二系統に分かれる。

Z 型 (6 本): $Z_1 Z_2, Z_2 Z_3, Z_4 Z_5, Z_5 Z_6, Z_7 Z_8, Z_8 Z_9,$

X 型 (2 本): $X_1 X_2 X_3 X_4 X_5 X_6, X_4 X_5 X_6 X_7 X_8 X_9.$

各ブロックの内側で隣り合う量子ビットの一致を見る Z 型がビット反転を捕まえ、三つのブロックをまたいで位相の食い違いを見る X 型が位相反転を捕まえる。誤りが二種類 (X と Z) あるから生成子も二系統要る、という構造が、生成子の顔ぶれにそのまま出ている。(生成子の並べ方・番号の取り方には流儀があり、本書では標準的な形で示す。確定した規約は以降の章で固定する。)

符号空間の中で論理状態を移す演算子 — 論理演算子 — は、この S を使って特徴づけられる。求めるのは、生成子をすべて測ったあとも壊れずに残り、しかも符号空間の上で恒等でない演算子だ。前者は「 S のすべての元と可換」、後者は「 S には属さない」と言い換えられる。 S と可換なパウリ演算子全体を正規化群 $N(S)$ と呼べば、論理演算子は $N(S)$ から S を除いた残り、 $N(S) \setminus S$ である。三量子ビット片で言えば、 $X_1 X_2 X_3$ は $Z_1 Z_2 \cdot Z_2 Z_3$ と可換 (命題 6.1 の重なりがそれぞれ偶数桁) で S には入らないから論理 X 、 Z_1 は同様に論理 Z になる。二つは桁を一つだけ共有して反可換 — $\{X_L, Z_L\} = 0$ — この反可換性は、符号空間の上に一個分のパウリ代数がそっくり乗っていることを意味する。論理演算子は S を掛けても作用が変わらない (X_L と $X_L \cdot (S \text{ の元})$ は符号空間の上で同じ) から、代表の選び

方には自由がある。この「 S を掛けても変わらない」自由が、第 3 章の縮退と同じ根を持つことを、最後に見る。

いよいよ、本章の二つ目の問い —— 測ってもデータが壊れない —— に構造で答える。生成子 g を測る操作は、その ± 1 固有空間への射影 $(1 \pm g)/2$ を状態に施すことだ。

定理 6.3 (測定保存) $g \in S$, $L \in N(S)$ とすると、定義から $[g, L] = 0$ である。射影子 $(1 \pm g)/2$ は g と恒等だけでできているので、 L はこの射影子とも可換になる：

$$\frac{1 \pm g}{2} \cdot L = L \cdot \frac{1 \pm g}{2}.$$

ゆえに測定 (= 射影) と論理操作 (= L) は順番を入れ替えられ、生成子を測っても論理状態に L が及ぼす作用は前後で変わらない —— 測定は論理情報を回さない。

これが測定保存の核心だ。実際、誤りのない符号語 $|\psi\rangle$ はすべての g の $+1$ 固有空間にいたので $(1 + g)/2 |\psi\rangle = |\psi\rangle$ 、測定は必ず $+1$ を返してこの状態に触れない。どこかに誤り E が乗って $E|\psi\rangle$ になり、 E が g と反可換なら、状態は g の -1 固有空間へ移って測定は -1 を返す。この -1 —— これがシンドロームだ。測定が読み取るのは「 g と反可換な誤りが乗ったか」だけで、符号語に書き込まれた論理の中身 $(a|0_L\rangle + b|1_L\rangle)$ の a, b には、可換性ゆえに指一本触れない。第 3 章で「スタビライザは論理的な自由に触れない作りだから測ってもデータを壊さない」と言葉で述べたことは、この $[g, L] = 0$ の一行に畳まれていたのである。

最後に、第 3 章の縮退を群の言葉で回収する。その前に一つ、演算子の意味が符号に対して相対的だという点に立ち止まる価値がある。観察の三量子ビット片では、 Z_1 は符号空間の上で論理 Z だった —— 論理状態を移す $N(S) \setminus S$ の演算子。ところが第 3 章で扱った 9 量子ビットの Shor 符号の中では、同じ Z_1 は訂正可能な位相誤りである。 S が大きくなり（八本の生成子）、外側 X 検査がブロック 1 の位相の食い違いを捕まえるようになると、ブロック 1 の Z は「直せる誤り」に変わる。同じ演算子 Z_1 が、属する S によって、論理演算子にも訂正対象にもなる。演算子に固定した意味があるのではなく、 S が意味を定めるのだ。

その Shor の中で、第 3 章の問いに答えが付く。実際には 2 番目に位相反転 Z_2 が起き、こちらは代表として Z_1 を当てて訂正したとする。訂正後の状態を決めるのは、起きた誤りと当てた訂正の差 —— 二つの積 —— である。

$$E_1^\dagger E_2 = Z_1 \cdot Z_2 = Z_1 Z_2 \in S.$$

命題 6.4 (縮退の群判定) 二つの訂正可能な誤り E_1, E_2 について、その差 $E_1^\dagger E_2$ が S に入っていれば、片方の訂正でもう片方を直せる（両者は符号空間の上で同じ作用をもつ）。

$Z_1 Z_2$ は、いまや名前のある対象だ —— Shor のスタビライザ群 S の生成子そのもの、 $Z_1 Z_2 \in S$ 。 S の元は符号空間を動かさないのだから、訂正後の状態はもとの符号語へ戻る。第 3 章で「符号空間を動かさない演算子の一つのまとまり」と呼んだそのまとまりに、ようやく S という名前が与えられ、縮退は「誤りどうしの差が S に入っているか」という一行の判定に整理された。

6.4 演習

(解答は付録 C に置く。数式の確認はすべて NumPy による実機実行で行うこと。)

■演習 6-1. パウリ演算子を $\text{GF}(2)$ 上のベクトルで表す。 n 量子ビットのパウリ演算子 P は、 X か Y の立つ桁を 1 とする X 成分ベクトルと、 Z か Y の立つ桁を 1 とする Z 成分ベクトルの対 $(x \mid z)$ で表せる。二つの演算子 $P = (x \mid z)$, $P' = (x' \mid z')$ が可換になるのはシンプレクティック内積 $x \cdot z' + z \cdot x' \pmod{2}$ が 0 のとき、反可換になるのは 1 のときである — これが重なり偶数則 (命題 6.1) の $\text{GF}(2)$ 版だ。 $Z_1 Z_2 \cdot Z_2 Z_3$ などいくつかの組についてこの内積を NumPy で計算し、可換/反可換が桁ごとの重なりの数え上げと一致することを確認せよ。さらに Shor の八生成子が互いにすべて可換であること (任意の二本でシンプレクティック内積が 0) を確認せよ。

■演習 6-2. 独立な生成子の集合から符号空間の射影子 $\Pi = \prod_i (1 + g_i)/2$ を組み、その階数 (= 符号空間の次元) が 2^{n-s} になることを数値で確かめよ。三量子ビット片 $\langle Z_1 Z_2, Z_2 Z_3 \rangle$ で $2^{3-2} = 2$ 、Shor の八生成子で $2^{9-8} = 2$ を確認する。生成子を一本ずつ加えるたびに階数が半分になる様子 ($8 \rightarrow 4 \rightarrow 2$) も見るとよい。

■演習 6-3. 与えられたパウリ演算子が論理演算子 $N(S) \setminus S$ に入るか — S のすべての生成子と可換で、かつ S には属さないか — を判定する手続きを書け。三量子ビット片で、 $X_1 X_2 X_3$ と Z_1 は論理演算子であること、 $Z_1 Z_2$ は S に属するので論理演算子でないこと、 X_1 は ($Z_2 Z_3$ とは可換だが $Z_1 Z_2$ と反可換なので) 符号空間を保たず論理演算子でないことを確かめよ。可換性は演習 6-1 のシンプレクティック内積で、 S への所属は生成子の積で張られるかで判定する。

第 7 章

CSS 構成と Steane 符号 —— 古典符号 2 枚から量子符号へ

7.1 片手の符号は、繰り返しでなくてよい

第 2 章の終わりに、一本の糸を残した。 X を直す 3 量子ビットの符号に、各量子ビットへ H を一枚ずつ当てると、 Z を直す符号がまるごと出てくる —— 片手 + 片手で両手、これが Shor の入れ子だった。そして書き添えた。 X を直す符号と Z を直す符号を、繰り返し符号に限らず一般の古典線形符号のペアから組み上げる構成があり、Steane 符号はそこから出てくる、と。本章はその糸を引く。

第 6 章で、私たちは符号を群の言葉で語れるようになった。スタビライザ群 S 、ビット反転を捕まえる Z 型生成子と位相反転を捕まえる X 型生成子、そして二つのパウリ演算子が可換になるのは反符号の重なりが偶数桁のときに限る、という重なり偶数則。本章で組み上げる符号も、この形をしている —— X 型と Z 型の生成子を持つアーベル部分群だ。問われているのは、その生成子をどこから持ってくるか、である。

Shor が使った片手落ちの符号 —— 3 量子ビットの繰り返し符号 —— は、正直に言って出来のよい古典符号ではない。3 ビットを費やして 1 ビット分の情報しか運べず、それで直せる誤りは 1 個。もっと割のよい符号はないのか。そして、繰り返しでない符号を持ってきたとき、第 2 章の「 H で写してもう片手」という処方はそのまま効くのか。一般に、二枚の古典符号を渡されたら、どんな条件のもとで、どんな生成子を持つ量子符号が組めるのか。

もう一つ問いがある。これから見る Steane 符号は、同じ $[7, 4, 3]$ Hamming 符号を二枚使って作る。なぜ二枚とも同じ符号なのか。同じものを二枚使うことに、利得があるのか。

行き先を先に置く。二枚の古典符号 —— X 型生成子を出す C_1 と Z 型生成子を出す C_2 —— に、一方の双対がもう一方に収まるというひとつの両立条件を課せば、 X 型生成子が C_1 の検査・ Z 型生成子が C_2 の検査になる量子符号が決まる。これが CSS 構成だ。そして二枚に同じ符号を選ぶと、 X 型と Z 型の生成子が同じ形になり、 X と Z を入れ替えても符号全体が動かなくなる —— この対称性が、軽い生成子と、後で見る素直な論理ゲートを生む。Steane は、その二枚に都合のよい $[7, 4, 3]$ Hamming を選んだ符号である。順に組み上げよう。

7.2 シンドロームが、反転したビットの番地を直に指す

まず、割のよい古典符号を一つ手に入れる。古典の世界で「1 個のビット反転を直す」のに、繰り返しはむしろ素朴すぎる道具だ。3 ビット繰り返しは 3 ビットで 1 ビットを運ぶ — 残り 2 ビットはまるごと冗長に消える。これに対し Hamming 符号 [7, 4, 3] は、7 ビットで 4 ビット分の情報を運びながら、やはり 1 個の誤りを直す。冗長はわずか 3 ビット、それで 4 ビットを保護する。同じ「1 誤り訂正」を、ずっと少ない割増しで実現するのだ（運べる情報の割合は、繰り返しの 1/3 に対し 4/7）。

Hamming 符号の仕掛けは、 3×7 の 0/1 行列 — 検査行列 $\mathbf{H}$ — にすべて畳まれている。

$$\mathbf{H} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

7 ビットの語 x （縦に並べる）に対し、 $s = \mathbf{H}x$ を mod 2 で計算した 3 ビットを**シンドローム**と呼ぶ。正しい語（符号語）は $s = 0$ を返し、どこか 1 ビットが反転すると s は 0 でなくなる。この $\mathbf{H}$ の妙は、列の並べ方にある — j 番目の列を、数 j の 2 進表示にしてあるのだ。1 列目は (0, 0, 1)、2 列目は (0, 1, 0)、…、7 列目は (1, 1, 1)。すると j 番目のビットだけが反転した語のシンドロームは、ちょうど $\mathbf{H}$ の j 列目、つまり j の 2 進表示そのものになる。シンドロームを 2 進数として読めば、それが反転したビットの番地を直に指す。

（厳密には、符号語 c の j 桁目だけを反転した語は $c \oplus e_j$ であり、 $\mathbf{H}c = 0$ だから $s = \mathbf{H}(c \oplus e_j) = \mathbf{H}e_j$ — 誤りベクトル e_j の寄与だけが残し、これが $\mathbf{H}$ の j 列目に等しい。）7 通りの 1 ビット誤りに、000 を除く 7 通りの 3 ビットパターンが、無駄なく一対一に対応する。これが「少ない割増しで効く」の中身だ。

この効率は、紙の上だけの話ではない。コンピュータのメモリが化けたビットを自力で直す ECC メモリは、Hamming 符号 — 正確には、もう 1 ビット足して 2 個の誤りの検出まで備えた拡張 Hamming (SECDED と呼ぶ) — を、いまでも使っている。繰り返して同じ保護をすれば冗長が嵩んで割に合わないところを、Hamming の効率が実用に乘せている。私たちがこれから量子符号の土台に据えるのは、現に動いている古典符号なのである。

（Hamming 符号そのものをどう作るか、なぜ列を 2 進表示にするとうまくいくのか、という古典符号理論の一般論には、本書では立ち入らない。ここで必要なのは「7 ビットで 4 ビットを運び 1 誤りを直す、検査行列 $\mathbf{H}$ を持つ符号がある」という事実だけだ。）

7.3 検査行列の各行が、そのまま生成子になる

ここで量子側に折り返す。第 2 章で、 X を直す符号に H を各量子ビットへ一枚ずつ当てると Z を直す符号が出ることを見た。やりたいのは、この二枚を — 今度は繰り返しでなく、良い古典符号 Hamming で — 入れ子にすることだ。第 6 章の言葉でいえば、ビット反転を見張る Z 型生成子と、位相反転を見張る X 型生成子を、両方そろえたい。古典の検査行列

は、まさに「誤りを見張る」ための行列だった。そこで素直にこうする —— 検査行列 $\mathbf{H}$ の各行を、 X を並べた演算子として読めば X 型生成子に、 Z を並べた演算子として読めば Z 型生成子になる。Steane 符号とは、同じ Hamming の検査行列を、一度は X として、もう一度は Z として、二枚重ねたものだ。

$\mathbf{H}$ の三行から、 X 型 3 本と Z 型 3 本、計 6 本の生成子が出る。各行で 1 が立つ桁に X (または Z) を置き、残りは恒等とする。

X 型	Z 型
$IIIXXXX$	$IIIZZZZ$
$IXXIIIX$	$IZZIIIZ$
$XIXIXIX$	$ZIZIZIZ$

第 6 章で Shor の 8 生成子を書き下したのと同じ流儀で、こんどは Hamming の三行から 6 本が立った。物理量子ビットは 7、生成子は 6、だから第 6 章の本数会計により $k = n - s = 7 - 6 = 1$ —— 論理 1 量子ビットを距離 3 で守る符号、 $[[7, 1, 3]]$ である。

ただし、6 本の演算子を書き並べたことと、それが「符号」を成すこととは、まだ別の話だ。第 6 章が教えたのは、スタビライザ符号とは生成子が共通に $+1$ で固定する空間であり、その空間が存在するためには生成子が互いに可換でなければならない、ということだった。可換でない二本を混ぜれば、両方を $+1$ に保つ状態は 0 に潰れてしまう。つまり、もし 6 本が互いに可換でなければ、いま並べたものは符号空間を持たず、Steane という構成そのものが空振りに終わる。可換性の確認は、形式上の儀式ではなく、この符号が成り立つかどうかの分かれ目なのだ。

幸い、確かめるべき相手は絞れる。 Z 型どうしは Z だけでできているから当然可換、 X 型どうしも同様。残るのは X 型と Z 型の組だけだ。ある Hamming 行 a に X を乗せた X 型生成子と、別の行 b に Z を乗せた Z 型生成子が可換になるのは、重なり偶数則により、二つの行 a と b が 1 を共有する桁の数が偶数のときに限る。

Hamming の三行を見ると、どの二行も —— 自分自身とも —— 偶数桁で重なっている。各行は 1 を 4 つ持ち (自己重なり 4)、相異なる二行はちょうど 2 桁で重なる。どちらも偶数だから、6 本はすべて互いに可換 —— Steane は符号空間を持つ。構成は空振りではなかった。

そして、この「偶数で重なる」は偶然ではない。Hamming 符号は、自分の双対符号を自分の中を含むという性質 —— 双対包含 $C^\perp \subseteq C$ —— を持つ。検査行列 $\mathbf{H}$ の各行は、双対符号 $\text{Hamming}^\perp$ を張るベクトルだ。双対包含が言うのは、これら双対側のベクトルが、Hamming のれっきとした語でもある、ということ。Hamming の語であれば、定義により $\mathbf{H}$ の検査をすべて通す —— $\mathbf{H}$ をかけて 0 になる —— から、 $\mathbf{H}$ のどの行とも直交する。直交とは、重なりが偶数ということだ。6 本の可換性は、この双対包含という一つの性質に根を持っていた。第 2 章で Shor の内側 Z 検査と外側 X 検査が偶数重なりで両立したのも、同じ根の特別な場合である。

これで Steane が群として立った。次節は、この組み方を一般の二枚に広げ、なぜ Steane が二枚とも同じ符号なのかを見る。

7.4 同じ符号を二枚使うと、 X と Z を入れ替えても符号は動かない

観察では、たまたま二枚とも同じ Hamming を使った。一般には、 X 型生成子を出す古典符号 C_1 と、 Z 型生成子を出す古典符号 C_2 を、別々に選んでよい。 C_1 の検査行を X に乗せれば X 型生成子 ($n - k_1$ 本)、 C_2 の検査行を Z に乗せれば Z 型生成子 ($n - k_2$ 本)。こうして組んだ符号を $CSS(C_1, C_2)$ 構成と呼ぶ。

二枚に課す両立条件は、すべての X 型と Z 型が可換であること — すなわち、観察で見た「検査行どうしが偶数で重なる」を、一般の二枚に言い換えたものだ。符号語の言葉で書けば $C_2^\perp \subseteq C_1$ (一方の双対がもう一方に収まる、双対包含)。この条件さえ満たせば生成子が同時に測れる量子符号が決まり、運べる論理量子ビット数は本数会計から $k = n - s = k_1 + k_2 - n$ となる。

Steane は、この一般構成で $C_1 = C_2 = [7, 4, 3]$ Hamming を選んだ場合だ。両立条件 $C_2^\perp \subseteq C_1$ は、観察で確かめた $\text{Hamming}^\perp \subseteq \text{Hamming}$ そのものになり、満たされている。論理量子ビット数も $k = 4 + 4 - 7 = 1$ で、観察で $n - s$ から数えた $[[7, 1, 3]]$ と一致する — 一般式が、具体的数え値をそのまま再現している。

二枚を同じ符号にしたことの利得が、ここで見えてくる。 C_1 と C_2 が同じなら、 X 型生成子と Z 型生成子は — X か Z かの違いを除いて — まったく同じ顔ぶれ (同じ Hamming 三行) になる。だから X と Z をいっせいに入れ替える操作をしても、生成子の集合 S は自分自身に移る。この「 $X \leftrightarrow Z$ で符号が動かない」性質を、自己双対 (self-dual) な CSS 符号と呼ぶ。(古典 Hamming 自身が自己双対なのではない — 自分の双対を含む、という一段弱い性質だ。その上に同じ符号を二枚重ねたとき、量子符号の側に X - Z 対称が立つ。)

この対称性は、まず生成子の weight に効く。第 6 章で、演算子の weight を「恒等でない桁の数 — 実際に手を触れている量子ビットの数」と決めた。Steane の生成子は、Hamming の検査行の 1 の数がそのまま weight になり、どの 6 本も weight 4 だ。比較に、第 6 章で書き下した Shor を見る。位相を見張る X 型生成子は $X_1 X_2 X_3 X_4 X_5 X_6$ のように 6 桁に触れる — weight 6。同じ距離 3 を守りながら、Steane の方が一回り軽い生成子で済んでいる。

この weight という量は、後で効いてくる。生成子が軽いほど、それを測るための補助の仕掛けが小さくて済み、測定そのものが新たに持ち込む誤りの種が減る — weight が小さいほど誤りに耐える敷居 (しきい値) が上がる、という話につながる。その定量は第 10 章に譲り、ここでは「Steane は Shor より軽い生成子を持つ」という量の差をつかんでおけばよい。weight は、符号の良し悪しを測る、距離とは別のもう一つのものさしなのだ。

対称性のもう一つの帰結は、論理ゲートの側に現れる。だがその前に — そもそも符号を組んだだけでは、まだ計算ができない。符号空間に納めた論理 1 量子ビットを操作する手立て、論理 0 を論理 1 へ反転させたり位相を付けたりする演算子が要る。それが論理 X と論理 Z だ。第 6 章は、それらが $N(S) \setminus S$ (S のすべてと可換だが S には属さない演算子) に住むと教えた — 可換だから測定で消えず、 S に属さないから符号空間の上で恒等にならな

い、まさに「壊さずに論理状態を動かす」演算子の条件である。Steane で最も素直な代表を探そう。

候補は、7 量子ビット全部に X を乗せた $X_L = X^{\otimes 7}$ と、全部に Z を乗せた $Z_L = Z^{\otimes 7}$ だ。 $X^{\otimes 7}$ は、 Z 型生成子 (weight 4) とは 4 桁で重なるから可換、 X 型生成子とは X どうしで可換、よって S 全体と可換。 X_L と Z_L は 7 桁すべてで重なるから互いに反可換 —— 符号空間の上に一個分のパウリ代数がそっくり乗る、第 6 章で見たあの構造だ。そして $X^{\otimes 7}$ も $Z^{\otimes 7}$ も S には属さない (全 1 のパターンは、weight 4 ばかりの Hamming[⊥] の語ではないからだ)。だから二つは $N(S) \setminus S$ の住人、れっきとした論理演算子である。以降の章は、この $X_L = X^{\otimes 7}$ 、 $Z_L = Z^{\otimes 7}$ を Steane の論理演算子の代表として使う。

論理演算子が決まれば、次に欲しいのは、それを実際にかける方法だ。理想を言えば、物理量子ビットに簡単な操作を一様にかけるだけで論理ゲートになってほしい —— そんな虫のいいことが起きるのか。自己双対は、その一例を与える。ここでは予告にとどめる。

各量子ビットに H を一枚ずつ当てる操作 $H^{\otimes 7}$ を考える。第 2 章で見たとおり H は X と Z を入れ替える。すると $H^{\otimes 7}$ は X 型生成子を Z 型生成子へ、 Z 型を X 型へ移す。ところが Steane は自己双対 —— X 型と Z 型が同じ顔ぶれ —— だから、入れ替えても生成子の集合 S は丸ごと自分に戻る。 S を保つ操作は符号空間を保ち、しかも $H^{\otimes 7}$ は $X_L = X^{\otimes 7}$ を $Z^{\otimes 7} = Z_L$ に移すから、これは論理アダマール H_L として働く。物理ゲートを一様に当てるだけで論理ゲートになる —— このことが、なぜ「単純さ」と「誤りに強いこと」を同時にもたらすのか、その仕組みは第 9 章で正面から扱う。本章では、自己双対という対称性が、軽い生成子とこの素直な論理ゲートの両方を生む、という一文を持ち帰ればよい。

行き先に着いた。二枚の古典符号 —— X 用 C_1 と Z 用 C_2 —— に、一方の双対がもう一方に収まる条件 $C_2^\perp \subseteq C_1$ を課せば、 X 型生成子が C_1 の検査・ Z 型生成子が C_2 の検査になる量子 CSS 符号が決まる。Steane は、その二枚に同じ $[7, 4, 3]$ Hamming を選んだ符号だ。同じ符号を二枚という選択が X - Z 対称 (自己双対) を生み、それが weight 4 の軽い生成子と、 $H^{\otimes 7}$ のような素直な論理ゲートをもたらす。第 6 章が符号に群を与え、本章がその群を古典符号二枚から組み上げる処方を与えた。残るのは、この符号の中身を開けることだ —— 論理 0 とは具体的にどんな状態で、なぜそういう形になるのか。次章はそこから始める。

7.5 演習

本章の各演習の解答は付録 C に置く。

演習 7-1 (Hamming のシンδροームが誤りの番地を指すこと) 検査行列 $\mathbf{H}$ (本文の 3×7) を NumPy で用意し、7 通りの 1 ビット誤り e_j (j 番目だけ 1 の縦ベクトル、 $j = 1, \dots, 7$) について $s = \mathbf{H}e_j$ を mod 2 で計算せよ。各シンδροームが、ちょうど j の 2 進表示 (= $\mathbf{H}$ の j 列目) になること、そして 7 つのシンδροームが 000 を除く 7 通りの 3 ビットパターンを一度ずつ取ることを確認する。これが古典 Hamming の復号 —— シンδροームを 2 進数に読んで反転ビットを名指す —— の中身だ。(第 I 部の Notebook とは独立に、この章だけで閉じる実装でよい。)

演習 7-2 (双対包含 $\text{Hamming}^\perp \subseteq \text{Hamming}$ の確認) $\mathbf{H}$ の三行が張る空間 — 双対符号 $\text{Hamming}^\perp$ ($[7, 3]$ simplex, $2^3 = 8$ 語) — を $\text{GF}(2)$ 上で生成し、その各語 v が Hamming の語でもあること、すなわち $\mathbf{H}v = 0 \pmod{2}$ を満たすことを確認せよ。同じことは、行どうしの直交 $\mathbf{H}\mathbf{H}^\top = 0 \pmod{2}$ を直接計算しても見える。この $\mathbf{H}\mathbf{H}^\top = 0$ が、本文で 6 本の生成子が互いに可換になる根拠だった。

演習 7-3 (検査行列から生成子を組み、可換性を数値で確かめる) 演習 6-1 の $\text{GF}(2)$ シンプレクティック表現 (X 成分・ Z 成分の対) を使い、 $\mathbf{H}$ の三行から X 型 3 本・ Z 型 3 本を構成せよ。6 本の任意の二本についてシンプレクティック内積を計算し、すべて 0 (互いに可換) になることを確認する。 X 型と Z 型の組では、この内積が Hamming 行どうしの重なり数の偶奇に一致することも見るとよい。さらに余裕があれば、 $X_L = X^{\otimes 7} \cdot Z_L = Z^{\otimes 7}$ が 6 本の生成子すべてと可換で、互いには反可換になることを確かめ、論理演算子の代表になっていることを確認せよ。

第 8 章

論理状態と論理演算子

8.1 論理 0 は、七つの 0 ではない

前章の終わりに、符号は群として立った。Steane の六本の生成子が共通に $+1$ で固定する空間が符号空間であり、そこに論理一量子ビットが一個分だけ納まる——物理七、論理一、距離三の $[[7, 1, 3]]$ である。だが「納まる」と言っただけで、中身はまだ開けていない。論理 0 とは、七個の物理量子ビットがどんな状態になっていることなのか。それを具体的に書き下すのが、本章の仕事だ。

素朴に考えれば、答えは決まっているように見える。論理 0 を蓄えたいなら、七個の量子ビットを全部 0 にすればよい—— $|0000000\rangle$ 。情報を七個へ冗長に広げる前の、いちばん素直な置き方だ。これが論理 0 だろう、と。

ところがこの置き方は、符号からこぼれ落ちる。符号空間に入る条件は、六本の生成子すべてに $+1$ で固定されることだった。 $|0000000\rangle$ を Z 型生成子で測れば、 Z は $|0\rangle$ を $+1$ で返すから、確かに $+1$ ——ここまではよい。だが X 型生成子はビットを反転させる演算子だ。ある X 型生成子 X^v を $|0000000\rangle$ に当てれば、 v の立つ桁が反転して別の基底状態 $|v\rangle$ に飛んでしまう。

$$X^v |0000000\rangle = |v\rangle \neq |0000000\rangle \quad (v \neq 0).$$

元と違う状態に移るのだから、 $|0000000\rangle$ は X 型生成子の $+1$ 固有状態ではない。六本のうち三本 (X 型) に固定されていない。 $|0000000\rangle$ は、そもそも符号語ですらないのだ。

ここに、誤り訂正符号の論理状態の核心がある。生成子の一部はビットを反転させる演算子であり、その反転で「動かない」状態でなければ符号には入れない。そして単一の基底状態は、反転すれば必ず別の状態へ動く。だから論理 0 は一つの基底状態ではありえず、反転しても自分に戻るよう、複数の基底状態へ広げた重ね合わせでなければならない。論理 0 を蓄えるとは、七個を全部 0 にすることではなく、ある特定の重ね合わせに置くことなのである。

ではその重ね合わせは、具体的にどんな状態か。何個の基底状態を、どう重ねれば、六本すべてに $+1$ で固定された状態が立つのか。論理 1 はそこからどう作り、論理 0 とどう違うのか。そして、その論理状態を壊さずに動かす論理演算子は、どう特定すればよいのか。本章は、まず符号化で $|0_L\rangle$ を具体物として取り出し、そこに現れる項の数がどこから強制されるのかを次に掘り下げ、最後に論理演算子の代数を確かめる。前章で組み上げた符号の、中身をいま

開ける。

8.2 論理 0 を取り出すと、八つの番地に散らばる

論理 0 は、 X 型生成子の反転で動かない重ね合わせでなければならない——前節でそこまで来た。では、その状態をどう手に入れるか。手がかりは、ほしい性質そのものにある。求めるのは「三本の X 型生成子すべてに $+1$ で固定される状態」だ。ならば適当な出発点を取り、それを X 型生成子たちで「均して」しまえばよい。

出発点には、さっき符号からこぼれた $|0000000\rangle$ を使う。これに X 型生成子を当てると別の番地へ飛ぶ——それがこぼれた理由だった。だが飛び先もろとも全部足し合わせて対称にすれば、こんどは X 型生成子で当てても顔ぶれが入れ替わるだけで全体は動かなくなる。具体的には、三本の X 型生成子を作る積の全パターンを $|0000000\rangle$ に施し、その行き先をすべて重ね合わせる。

三本の X 型生成子は、それぞれ Hamming 検査行列の一行に X を乗せたものだった（前章）。その積で作れるパターンは、検査行の $\text{GF}(2)$ 線形結合——すなわち双対符号 $C^\perp$ （前章 Hamming 符号 $C = [7, 4]$ の双対であり、演習 7-2 で数えた $[7, 3]$ simplex 符号）の語そのものだ。各パターン X^v を $|0000000\rangle$ に当てれば $|v\rangle$ に移るから、重ね合わせは八つの基底状態の均等な和になる。

$$\begin{aligned} |0_L\rangle &= \frac{1}{\sqrt{8}} \sum_{v \in C^\perp} |v\rangle \\ &= \frac{1}{\sqrt{8}} (|0000000\rangle + |0001111\rangle + |0110011\rangle + |0111100\rangle \\ &\quad + |1010101\rangle + |1011010\rangle + |1100110\rangle + |1101001\rangle). \end{aligned}$$

八項の内訳は、重み 0 の $|0000000\rangle$ がひとつと、重み 4 の語が七つ。これらは前章で「 $C^\perp \subseteq C$ （双対包含）」を確かめたときに $\text{GF}(2)$ 上で生成した、まさにあの八語だ。論理 0 は、その八つの番地に均等に散らばった重ね合わせだった——そして素朴な予想 $|0000000\rangle$ は、まったくの間違いではなく、この八項のうちの一項にすぎなかったのである。（なぜちょうど八で、ひとつでも二つでもないのかは一般化で正面から扱う。ここでは「均す」操作の行き先がこの八語になる、という事実を押さえる。）

組み上げた状態が本当に符号空間に入っているか——六本の生成子すべてに $+1$ で固定されるか——を確かめよう。 X 型生成子は、いま見たとおり八項の顔ぶれを並べ替えるだけだから、状態を動かさず $+1$ を返す（対称にした目的がこれだった）。残るは Z 型生成子だ。 Z 型生成子 Z^w （ w も Hamming 検査行）を八項に当てると、各 $|v\rangle$ には符号 $(-1)^{w \cdot v}$ がつく。ところが w も v も双対符号 $C^\perp$ の語であり、前章で見た双対包含は「 $C^\perp$ の語どうしは偶数桁で重なる」を意味する——つまり $w \cdot v$ は偶数、符号は必ず $+1$ だ。だから Z 型生成子も八項をそのまま素通りさせる。六本すべてが $+1$ 。 $|0_L\rangle$ は確かに符号空間の住人である。

ここで効いたのは、前章で六生成子の可換性を支えたのと同じひとつの性質——双対包含だった。あのときは「 X 型と Z 型が両立して同時に測れる」根拠であり、いまは「 Z 型生成

子が論理 0 を素通りさせる」根拠になっている。ひとつの性質が、符号が立つことと、その符号語が固定されることの両方を支えている。

8.3 論理 1 は全ビットを裏返した八項、測れば偶奇で見分けがつく

論理 0 が立った。論理 1 はどう作るか。論理状態を 0 から 1 へ反転させる演算子は、前章で代表を決めてあった——論理 X は $X_L = X^{\otimes 7}$ 、七量子ビット全部に X を一枚ずつ当てる演算子だ（各量子ビットに一樣に当てる、transversal な操作。なぜ一樣に当てるだけで論理ゲートになるのかという仕組みは次章に譲り、ここでは前章で定めた代表をそのまま使う）。これを $|0_L\rangle$ に施す。 $X^{\otimes 7}$ は各番地のビットをすべて裏返すから、八項のひとつひとつ $|v\rangle$ が、その全ビット反転 $|v \oplus 1111111\rangle$ に移る。

$$|1_L\rangle = X^{\otimes 7} |0_L\rangle = \frac{1}{\sqrt{8}} \sum_{v \in C^\perp} |v \oplus 1111111\rangle.$$

できあがる八語は、 $|0_L\rangle$ の八語の全ビット補集合だ。重み 0 の $|0000000\rangle$ は重み 7 の $|1111111\rangle$ に、重み 4 の七語は重み 3 の七語に移る。つまり論理 1 は、重み 7 がひとつと重み 3 が七つ——すべて奇数の重みの番地に散らばる。論理 0 が偶数の重みばかり、論理 1 が奇数の重みばかり。両者はひとつの基底状態も共有しないから内積は 0、確かに直交する。 $X_L = X^{\otimes 7}$ を全ビット反転の代表に選んだことで、論理 1 が論理 0 の素直な裏返しとして書けた（前章で最小重みの代表を避けた理由が、ここで形になる）。

直交を、式の上だけでなく測定の手続きで確かめておこう。いちばん軽いやり方は、七量子ビットを全部 Z 基底で測り、出た 0/1 列のパリティ——1 の個数の偶奇——を読むだけ、というものだ。なぜそれで論理ビットが読めるのか。論理 Z は $Z_L = Z^{\otimes 7}$ 、七桁すべての Z の積であり、その固有値は測定列のパリティにほかならない。 $|0_L\rangle$ を測れば、出る列は必ず偶数重みの八語のどれか → パリティ偶 → $Z_L = +1$ → 論理 0。 $|1_L\rangle$ なら必ず奇数重み → パリティ奇 → $Z_L = -1$ → 論理 1。シンδροームも訂正もいらず、パリティを直に読むだけで二つが見分けられる。

ただし、これは誤りが何も起きていない理想の場合の話だ。途中で物理量子ビットに誤りが乗れば、測定列はもう八語に収まらず、パリティを直読みするだけでは論理ビットを取り違える。誤り込みで正しく論理ビットを読むには、まずシンδροームを取って誤りを訂正してから論理パリティを読む「フル版」の論理測定が要る——それは transversal の機構と合わせて次章で扱う。本章では、誤りなしで二つの論理状態が直交することを、この軽い測定で確かめておけば足りる。

これで論理 0、論理 1、そしてそれを見分ける軽い測定まで、具体物として手元に揃った。残っているのは、最初に立てた問いの芯だ——なぜ論理 0 はちょうど八項で、ひとつでも二つでもいけなかったのか。次節でそこを正面から掘る。

8.4 八項は選んだのではなく、三つの独立な反転に強いられた

論理 0 が八項になることは、観察で手を動かして確かめた。だが「八という数はどこから来たのか、なぜひとつでも二つでもいけないのか」——最初に立てた問いの芯には、まだ答えていない。ここを正面から見る。

まず、八という数の出どころははっきりしている。論理 0 は三本の X 型生成子で $|0000000\rangle$ を均したものだ。三本の生成子は互いに独立——どれも他の二本の積では作れない（積で作れるなら本数が減り、 $k = n - s$ の勘定が狂う。前章で六本の独立性を要求したのは、まさにこのためだ）。互いに可換で、二乗すれば恒等に戻る独立な反転が三つ。これが生む積のパターンは、各生成子を「掛ける／掛けない」の二択の組み合わせ—— $2 \times 2 \times 2 = 2^3 = 8$ 通りだ。均す操作はこの八通りの行き先をすべて足し合わせるから、項の数はちょうど八。一般に独立な X 型生成子が m 本あれば論理 0 は 2^m 項になる。Steane は X 型が三本 ($n - k_1 = 7 - 4 = 3$) だから、 $2^3 = 8$ 項なのである。

ではなぜ、もっと少なくできないのか。鍵は、論理 0 に課された条件——三本の X 型生成子すべてに $+1$ で固定される——を、状態の側から読み直すことにある。 X 型生成子 X^v ($v \neq 0$) に $+1$ で固定されるとは、 $X^v |\psi\rangle = |\psi\rangle$ 、すなわち v の桁を反転させても状態が変わらない、ということだ。

ここで単一の基底状態 $|b\rangle$ を当てはめてみる。 $X^v |b\rangle = |b \oplus v\rangle$ 。反転すれば必ず別の番地に移るのだから、一項では決して X^v に固定されない。一項は、 X 型生成子が一本でもあれば失格だ——これは問い段で見た通り。

では二項ならどうか。 $|b\rangle$ と $|b \oplus v\rangle$ を等しく重ねた対称ペアは、 X^v で入れ替わるだけだから、確かに X^v には固定される。だが論理 0 が応えねばならないのは一本ではなく、独立な三本すべてだ。一本目 g_1 への固定が $|b\rangle$ に相棒 $|b \oplus v_1\rangle$ を呼ぶ。二本目 g_2 への固定は、その二つそれぞれにさらに相棒 $|b \oplus v_2\rangle$ と $|b \oplus v_1 \oplus v_2\rangle$ を呼ぶ。三本目 g_3 がもう一度倍にする。三本が独立だから——どの反転も他の反転からは届かない番地を指すから——呼び込みは一度も重ならず、番地は $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$ と倍々に増えて、八で初めて閉じる。途中のどの段でも止まらない。だから論理 0 は八項を下回れない。八は、最小であると同時に必要な数なのだ。

この「均して固定する」操作には、前章までに用意した道具で正確な形を与えられる。三本の生成子すべてに $+1$ で固定される成分を取り出す射影子は、第 6 章で見た

$$\Pi = \prod_i \frac{1 + g_i}{2}$$

だった。三本ぶんの積を展開すると、ちょうど八個の項（八通りの生成子の積）の和になり、それを $|0000000\rangle$ に施せば八つの番地が立ち上がる。射影子の項の数が八であること、それがそのまま論理 0 の項の数になる。観察で「均す」と言った操作は、この射影子のことだった。

八項は、見栄えで選んだ数ではない。独立な三つの反転に同時に固定されよ、という条件だけから、最小かつ必要な数として八が決まる。本編で「裏話」として触れたこの事実が、いま条件から導かれた。

8.5 論理ビットは、符号空間の上でひとつのパウリ代数を回す

論理 0 と論理 1 が具体物として立った。残る問いは三つめ——論理状態を壊さずに動かす演算子を、どう特定するか、だ。

答えの枠組みは第 6 章にあった。論理演算子は $N(S) \setminus S$ ——スタビライザ群 S のすべてと可換だが、 S 自身には属さない演算子——に住む。 S と可換だから、生成子を測っても固有値に現れず、論理状態を保つ。 S に属さないから、符号空間の上で恒等ではなく、ちゃんと状態を動かす。前章で代表に選んだ $X_L = X^{\otimes 7}$ と $Z_L = Z^{\otimes 7}$ が、この条件を満たすことは前章で確かめた。

この二つが符号空間の上でどう振る舞うかを見よう。 $X^{\otimes 7}$ と $Z^{\otimes 7}$ は七つの桁すべてで重なる。各桁では X と Z が反可換だから、七つぶん——奇数個——の符号がかかって、全体としては反可換になる。 $X_L Z_L = -Z_L X_L$ 。これは、裸の一量子ビットで X と Z が織りなすのと同じ反交換関係だ。実際、 X_L は $|0_L\rangle$ と $|1_L\rangle$ を入れ替え（観察で見た）、 Z_L は $|0_L\rangle$ に $+1$ 、 $|1_L\rangle$ に -1 を返す（軽い測定がそれだった）。符号空間という七量子ビットの片隅に、一量子ビットぶんのパウリ代数がそっくり乗っている。論理 0/1 は、ただ直交する二状態であるだけでなく、裸の量子ビットと同じ演算規則で操れる、本物の一量子ビットなのだ。

裸の量子ビットなら、計算基底 $\{|0\rangle, |1\rangle\}$ のほかに、その重ね合わせの基底 $\{|+\rangle, |-\rangle\}$ があった。論理ビットにも同じものを置ける。

$$|+_L\rangle = \frac{|0_L\rangle + |1_L\rangle}{\sqrt{2}}, \quad |-_L\rangle = \frac{|0_L\rangle - |1_L\rangle}{\sqrt{2}}.$$

X_L は $|0_L\rangle$ と $|1_L\rangle$ を入れ替えるから、その和 $|+_L\rangle$ は X_L で動かない ($X_L |+_L\rangle = |+_L\rangle$)、論理 X の $+1$ 状態)、差 $|-_L\rangle$ は符号が反転する (-1 状態)。裸の量子ビットで $|+\rangle$ が X の $+1$ 固有状態だったのと、そっくり同じ構図だ。これで論理ビットは、計算基底 $\{|0_L\rangle, |1_L\rangle\}$ と重ね合わせ基底 $\{|+_L\rangle, |-_L\rangle\}$ の両方を備えた、過不足のない一量子ビットになった。

ついでに、 $|+_L\rangle$ を物理量子ビットの番地で書き下すと、もうひとつ前章との対応が見える。 $|0_L\rangle$ は双対符号 $C^\perp$ の八語に散っていた。その全ビット反転の $|1_L\rangle$ を足すと、 $C^\perp$ の八語と、それを反転した八語——合わせて Hamming 符号 C そのものの十六語——に均等に散った状態になる。論理 0 が双対符号 $[7, 3]$ の上に、論理 $+$ が元の Hamming 符号 $[7, 4]$ の上に乗る。CSS 構成で二枚貼りした古典符号が、論理状態の形にそのまま顔を出している。

最後に、論理演算子の特定について一つ注意を残す。論理 X として $X^{\otimes 7}$ を選んだが、これは唯一の選択ではない。 $X^{\otimes 7}$ に S の元を掛けた演算子——例えば前章で触れた、もっと軽い重み 3 の代表——は、符号空間の上で $X^{\otimes 7}$ とまったく同じ作用をする。 S の元は符号語を $+1$ で素通りさせるから、掛けても論理状態への効き目は変わらないのだ。だから「論理 X 」とは一個の演算子ではなく、 S を法として等しい演算子たちの一族であり、 $X^{\otimes 7}$ はそのうち最も対称な代表にすぎない。 $N(S) \setminus S$ の「割る」が、ここで具体的な意味を持つ。どの代表を選ぶかは、次章で論理ゲートを物理操作として実装するとき効いてくる——一様かけられる $X^{\otimes 7}$ が、実装の上でとりわけ都合がよいのである。

これで、前章で組み上げた符号の中身がすべて開いた。論理 0 と論理 1 はそれぞれ八つの番地に散った具体的な重ね合わせであり、その八は独立な三つの反転から強いられた最小の数だった。論理 $X \cdot$ 論理 $Z \cdot$ 論理 $+$ が符号空間の上に一量子ビットぶんのパウリ代数を回し、論理演算子は S を法として代表に自由だった。だが、ひとつ未払いの約束が残っている——論理 X を作るのに $X^{\otimes 7}$ を「全量子ビットに一様にかけるだけ」で済ませた、あの手軽さだ。なぜ物理ゲートを一様にかけるだけで論理ゲートになるのか。そしてなぜ、その操作は誤りを増やさないのか。単純さと誤りへの強さが同じ仕掛けの両面である、という次章の主題が、ここから始まる。

8.6 演習

以下の演習の解答は付録 C にまとめる。すべて理想実行（誤りなし）に絞る。

演習 8-1. 論理 0 を組み、八項と全スタビライザ $+1$ を確かめる。演習 7-2 で生成した双対符号 $C^\perp$ の八語を使い、 $|0_L\rangle = \frac{1}{\sqrt{8}} \sum_{v \in C^\perp} |v\rangle$ を七量子ビットの状態ベクトルとして NumPy で構成せよ（あるいは $|0000000\rangle$ に三本の X 型生成子の射影子 $\prod_i \frac{1+g_i}{2}$ を施しても同じ状態が出る。両者の一致を確かめてもよい）。得られた状態が八つの基底状態にのみ非零の振幅を持つこと、その八つが重み 0 の語ひとつと重み 4 の語七つであることを確認する。続いて、前章で構成した六本の生成子それぞれについて期待値 $\langle 0_L | g | 0_L \rangle$ を計算し、すべて $+1$ になる—— $|0_L\rangle$ が符号空間の住人である——ことを確かめよ。

演習 8-2. 論理 1 を作り、軽い論理測定で直交を確かめる。 $|0_L\rangle$ に $X^{\otimes 7}$ を施して $|1_L\rangle$ を作り、その八項が $|0_L\rangle$ の各項の全ビット反転（重み 7 ひとつと重み 3 七つ、すべて奇数重み）になっていることを確認せよ。次に「軽い論理測定」を実装する： $|0_L\rangle$ と $|1_L\rangle$ のそれぞれを七量子ビットすべて Z 基底で測り、出る各 0/1 列の 1 の個数の偶奇を読む。 $|0_L\rangle$ からは必ず偶数（ $Z_L = +1 =$ 論理 0）、 $|1_L\rangle$ からは必ず奇数（ $Z_L = -1 =$ 論理 1）になり、訂正なしで二つが見分けられることを確かめる。最後に内積 $\langle 0_L | 1_L \rangle = 0$ を直接計算し、両者が直交することを確認せよ。（誤りを乗せた場合にこの軽い測定が破れることは次章のフル論理測定で扱う。本演習は理想実行に絞る。）

演習 8-3. 論理 $+$ を置き、論理ビットがひとつのパウリ代数を回すことを確かめる。 $|+_L\rangle = (|0_L\rangle + |1_L\rangle)/\sqrt{2}$ を構成し、 $X^{\otimes 7}$ （論理 X ）の $+1$ 固有状態 $X^{\otimes 7} |+_L\rangle = |+_L\rangle$ になっていることを確認せよ。あわせて $X_L = X^{\otimes 7}$ と $Z_L = Z^{\otimes 7}$ が反可換（ $X_L Z_L = -Z_L X_L$ ）であり、符号空間の上で $|0_L\rangle / |1_L\rangle$ を一量子ビットのパウリのように動かすことを確かめる。〔発展〕 $|+_L\rangle$ を計算基底で書き下すと元の $[7, 4]$ Hamming 符号の十六語に均等に散ること（ $|0_L\rangle$ が双対符号 $[7, 3]$ の八語に散ったのと対をなす）を確認する。〔発展〕 $X^{\otimes 7}$ に何か一本の生成子を掛けた演算子が符号空間の上で $X^{\otimes 7}$ と同じ作用をすること（＝同じ論理 X の別代表）を確かめ、論理演算子が S を法として代表に自由であることを数値で見る。

第 9 章

transversal ゲートとフォールトトレランス — 単純さと耐性が同じ仕掛け

9.1 一様な操作が、論理ゲートになり、しかも誤りを増やさない

第 7 章の終わりに、予告を一つ残した。Steane の各量子ビットに H を一枚ずつ当てる操作 $H^{\otimes 7}$ は、 X 型生成子を Z 型へ、 Z 型を X 型へ入れ替える。ところが Steane は自己双対 — X 型と Z 型が同じ顔ぶれ — だから、入れ替えても生成子の集合 S は丸ごと自分に戻り、しかも $X_L = X^{\otimes 7}$ を $Z_L = Z^{\otimes 7}$ へ移す。だからこれは論理アダマール H_L として働く、と。物理ゲートを一様に当てるだけで論理ゲートになる、その仕組みを正面から扱う — そう書いて章を閉じた。本章がその約束を引き取る。

落ち着くと、これは妙な話だ。論理アダマールは、符号空間に納めた論理一量子ビットを回す、れっきとした論理操作である。その論理操作を、七個の物理量子ビットに — 符号の構造など知らぬげに — ただ H を一律に配るだけで実現できる、という。手品のように見える。なぜ、符号を気にかけない一様な手つきが、ちょうど論理ゲートに化けるのか。

もう一つ、同じくらい妙なことがある。第 I 部第 4 章で、私たちは訂正装置自身の故障を恐れた。誤りは CNOT を伝って増える — X は制御から標的へ、 Z は標的から制御へ — から、一個の故障が二個三個の誤りに育てば、距離 3 の符号はもう直せない。第 4 章の結論は、fault-tolerant とは抽象的な正しさではなく具体的な回路の組み方であり、「一個の故障が、ブロックの中で高々一個の誤りにしか広がらない」ことだった。では、いま見ている一様な操作は、この危ない増殖に対してどうなのか。論理ゲートをかけるたびに誤りが増えるなら、せっかく組んだ符号は計算の途中で崩れてしまう。

先に、本章の見立てを置く。この二つの妙 — 単純さと、誤りに強いこと — は、別々の幸運ではない。どちらも、transversal という一つの構造から出る。transversal とは、物理ゲートが一量子ビットだけに閉じる ($H^{\otimes 7}$ のように一様に配る) か、別のブロックの相方とだけ組む (後で見るブロック間 CNOT) かで、同一ブロックの二量子ビットに同時には決して触れない、という組み方のことだ。この「同一ブロックの二点に触れない」が、一様にかけるだけという単純さを生み、同時に、一個の故障が同一ブロックで二個に育つ道を塞ぐ。単純さと耐性は、同じ仕掛けの両面なのである。

ただし、もう一段の区別が要る。transversal な操作が「誤りを増やさない」のは locality だけで決まり、ゲートの中身に依らない — どんな transversal ゲートでもそうだ。だが、その単純な操作が「論理ゲート」になるかどうかは別問で、生成子をどう動かすかという代数の条件が要る。 $H^{\otimes 7}$ がたまたま論理アダマールになったのは、Steane の自己双対がその条件を満たしたからだった。この二つを分けて見ることが、本章を通すと同時に、なぜ transversal で作れるゲートに天井があるのか（第10章）の入口になる。

そして最後に、誤り込みで論理ビットを読む手続きを見る。論理ゲートで計算した後、答えを取り出すには論理状態を測らねばならない。だが物理量子ビットには誤りが乗っているかもしれない。誤りを抱えたまま測って、なお正しい論理ビットを読むには、どうするか。これも、第7章で組んだ古典 Hamming のデコーダがそのまま効く、という話になる。

行き先を先に置く。第一に、一様な $H^{\otimes 7}$ が論理アダマールになるのは、それが S を S に写し X_L を Z_L に移すからで、手品ではない。第二に、ブロック間の transversal CNOT は、制御側の一個の誤りを標的側のたった一個へ送るだけで、同一ブロックの兄弟には漏らさない — だから一個の故障が一個の誤りで止まる。第三に、全量子ビットを測ってシンドロームを取り、表で誤りを直してから論理パリティを読む手続きは、古典デコードそのものだ。そしてこの三つを貫くのが transversal という一つの構造であり、それが単純さと耐性を同時に生む。順に見ていこう。

9.2 一様な H が論理アダマールになるのは、手品ではない

論理ゲートとは何かを、まずはっきりさせる。論理アダマールに求められるのは二つだ。第一に、符号空間を符号空間のまま保つこと — 論理状態を、符号の外（誤りのある状態）へ蹴り出してはならない。第二に、論理演算子に正しく作用すること — アダマールなら、論理 X と論理 Z を入れ替えることだ。この二つを満たして初めて、その操作は論理アダマールと呼べる。

第一の条件を、群の言葉に翻訳する。第6章で、符号空間とはスタビライザ群 S が共通に $+1$ で固定する空間だった。操作 U をかけた後の状態が、なお符号空間にいる — 各生成子 g で測ってやはり $+1$ が返る — ためには、 U が S をまるごと S 自身へ写せばよい。つまり、各生成子 g を U で共役した $UgU^\dagger$ が、また S の元であること、

$$UgU^\dagger \in S \quad (\text{すべての } g \in S)$$

である。こうであれば、 U をかけた状態は依然としてすべての生成子に $+1$ を返し、符号空間にとどまる。逆に、もし U がある生成子を S の外の演算子へ飛ばせば、その向きには $+1$ が保証されず、状態は符号からこぼれる。これが、第6章で名を伏せた「符号空間を動かさない操作」の、群による正体だ。

では $H^{\otimes 7}$ はこの条件を満たすか。第2章で見たとおり、 H は一量子ビット上で X と Z を入れ替える。七枚いっせいに当てる $H^{\otimes 7}$ は、各桁の X を Z に、 Z を X に替える。Steane の X 型生成子は、Hamming の検査行の立つ桁に X を置いたものだった（たとえば

$IIIXXXX$)。これに $H^{\otimes 7}$ を当てると、同じ桁に Z が立つ。

$$H^{\otimes 7} (IIIXXXX) (H^{\otimes 7})^\dagger = IIIZZZZ,$$

すなわち Z 型生成子になる。 X 型三本は、対応する Z 型三本へ移る。同じく Z 型三本は X 型三本へ移る。

ここで自己双対が効く。第 7 章で確かめたように、Steane の X 型生成子と Z 型生成子は、 X か Z かの違いを除いて同じ顔ぶれ —— 同じ Hamming 三行 —— だった。だから $H^{\otimes 7}$ が X 型を Z 型へ、 Z 型を X 型へ移しても、移った先はすべて元から S の中にいる生成子だ。生成子の集合 S は、丸ごと自分自身に戻る。

$$H^{\otimes 7} S (H^{\otimes 7})^\dagger = S.$$

第一の条件は満たされた。 $H^{\otimes 7}$ は符号空間を符号空間のまま保つ。

この戻りは、自己双対あつてのものだ。もし符号が自己双対でなく、 X 型生成子を H で写した Z 演算子が S の語でなかったら、 $H^{\otimes 7}$ は S を S の外へ蹴り出し、符号空間を保たない —— つまり論理ゲートにすらならない。第 7 章で「同じ符号を二枚使う」という選択が X - Z 対称を生んだ、あの一手が、ここで論理ゲートを一枚、無償で手渡している。

第二の条件も見ろ。論理演算子の代表は、第 7 章で $X_L = X^{\otimes 7}$ 、 $Z_L = Z^{\otimes 7}$ と決めた。 $H^{\otimes 7}$ を $X^{\otimes 7}$ に当てれば、各桁の X が Z になり、 $Z^{\otimes 7} = Z_L$ 。 Z_L に当てれば X_L 。

$$H^{\otimes 7} X_L (H^{\otimes 7})^\dagger = Z_L, \quad H^{\otimes 7} Z_L (H^{\otimes 7})^\dagger = X_L.$$

論理 X と論理 Z が、ちょうど入れ替わる —— これは論理アダマールの作用そのものだ。二つの条件がそろった。一様に配った $H^{\otimes 7}$ は、論理アダマール H_L として働く。

手品の種は割れた。一様な手つきが論理ゲートに化けたのは、偶然ではない。 $H^{\otimes 7}$ が生成子をどう共役するか (X 型 $\leftrightarrow$ Z 型) と、Steane の自己双対 (X 型と Z 型が同じ) とが噛み合って、 S を保ち論理演算子を正しく入れ替えることを、代数が強制している。符号の構造を知らぬげに H を配ったつもりで、実は符号の対称性が、その一様さをちょうど論理アダマールへと整えていたのだ。

ただし、いま示したのは「なぜ論理ゲートになるか」だけだ。 $H^{\otimes 7}$ は一量子ビットずつの操作だから、一個の故障はその一桁に留まり、誤りは増えようがない —— 耐性の側は、一量子ビット操作では当たり前すぎて見えにくい。耐性が本当に問われるのは、二つの量子ビットを絡める操作 —— CNOT —— をかけるときだ。次節はそこへ進む。

9.3 ブロック間の CNOT は、誤りを兄弟に渡さない

論理アダマールは、各量子ビットを独立にいじる一量子ビット操作だった。だから一個の故障はその一桁に閉じ、誤りの増殖は問題にならない。耐性が本当に試されるのは、二つの量子ビットを絡める操作 —— CNOT —— を論理レベルでかけるときだ。論理 CNOT は、二つの論理量子ビット、つまり七個ずつの物理量子ビットを持つ二つのブロックを、相互作用させる。ここで配線を誤れば、一個の故障が増殖する余地が生まれる。

論理 CNOT の組み方は、こうだ。制御の役を担うブロック A と、標的の役を担うブロック B を用意する。そして、 A の量子ビット i から B の量子ビット i へ、物理 CNOT を一本、 $i = 1$ から 7 まで平行に渡す。要点は、各物理 CNOT が触れるのは A の一個と B の一個 — 別々のブロックの相方どうし — だけで、同一ブロックの二つの量子ビットには決して同時に触れない、という点にある。

これが論理 CNOT になることは、前節と同じ筋で確かめられる。transversal CNOT は、 A 側の X 型生成子を「 A 側の生成子 $\times B$ 側の対応する生成子」へ、というように生成子を生成子（の積）へ写し、 S を S の中に保つ。そして論理演算子の上では、

$$X_{L,A} \mapsto X_{L,A} X_{L,B}, \quad Z_{L,B} \mapsto Z_{L,A} Z_{L,B}$$

のように、ちょうど CNOT が論理ビットに対してすべき作用を実現する（詳しい共役の一覧は一般化に回す）。前節で見た「 S を保ち、論理演算子に正しく作用する」という論理ゲートの条件を、これも満たしている。

本節の主役は、この CNOT が誤りをどう扱うかだ。第 I 部第 4 章の伝播則を、そのまま当てる — X は制御から標的へ、 Z は標的から制御へ。

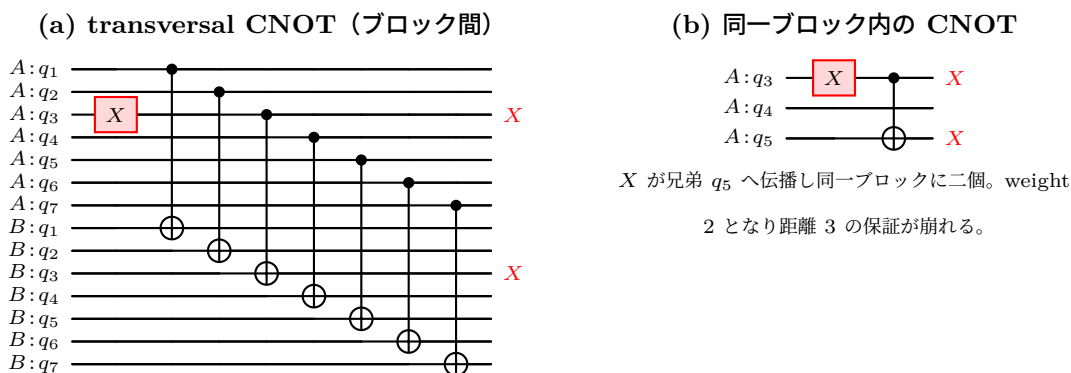
いま、論理 CNOT をかける直前に、制御ブロック A の量子ビット i に X 誤りが一個乗っていたとする。transversal な配線では、 A の量子ビット i は、 B の量子ビット i へ向かう物理 CNOT の制御だ。第 4 章の規則「制御の X は標的へ伝わる」がそのまま効く。 A の量子ビット i の X は、CNOT を通り抜けて、 B の量子ビット i へ写る。操作の後、 A の量子ビット i に X 、 B の量子ビット i に X が残る。

ブロックごとに数えよう。ブロック A は、もとの一個の誤り（量子ビット i の X ）をそのまま持つ — weight 1。ブロック B は、新たに一個（量子ビット i の X ）を受け取った — weight 1。各ブロックは、高々一個の誤りしか抱えていない。そして肝心なのは、兄弟が無傷だということだ。 A の他の量子ビット ($j \neq i$) にも、 B の他の量子ビットにも、誤りは一切漏れていない。誤りはブロックをまたいで一個、相方の量子ビットへ渡っただけで、同じブロックの中では広がらなかった。

なぜ兄弟に漏れないのか。 A の量子ビット i に触れる物理操作は、 B の量子ビット i へ向かう一本の CNOT だけだからだ。 A の量子ビット i から A の他の量子ビットへ向かう物理操作は、存在しない。だから A の量子ビット i の X が行ける先は B の量子ビット i ただ一つで、同じブロックの兄弟 A の量子ビット j へは、伝わりようがない。 Z 誤りでも同じだ。 B の量子ビット i に乗った Z は、第 4 章の規則「標的の Z は制御へ伝わる」により A の量子ビット i へ写る — これもブロックをまたいで一個、各ブロック weight 1 に収まる。

結論はこうだ。論理 CNOT の最中に物理的な故障が一個起きても、各ブロックに残る誤りは高々一個。これは第 I 部第 4 章で定めた fault-tolerance の定義 — 一個の故障は、各符号ブロックで高々一個の誤りにしか広がらない — を、そのまま満たしている。transversal な論理 CNOT は、定義上 fault-tolerant なのだ。誤りは確かに「広がった」が、広がったのはブロックの間であって、ブロックの中ではない。各ブロックは自分の符号で、自分の一個の誤りを独立に直せる。

この踏みとどまりが、配線の選び方に懸かっていることを、対比で確かめる。もし論理



故障 X は相方 $B:q_3$ へ一個渡るだけ。各ブロック weight 1、兄弟は無傷。

図 9.1: transversal CNOT は誤りをブロックの間に一個渡すだけで、同一ブロックの兄弟には伝えない。

CNOT を、transversal でない仕方でも — たとえば同じブロック A の中の二つの量子ビット、量子ビット 3 を制御、量子ビット 5 を標的にした物理 CNOT を使って — 組んだとしよう。すると、量子ビット 3 に乗った X は、規則どおり標的の量子ビット 5 へ伝わる。今度は写った先が、別のブロックではなく同じブロック A の兄弟だ。ブロック A は、量子ビット 3 と量子ビット 5 に X を二個抱える — weight 2。距離 3 の符号が確実に直せるのは weight 1 までで、weight 2 はもう直せない（別の単一誤りと見分けがつかなくなったり、論理演算子と取り違えたりする）。一個の故障が、同一ブロックで二個の誤りに育った — 第 4 章が名指した、まさにあの破滅だ。

transversal は、見栄えのための作法ではない。同一ブロックの二量子ビットに触れる物理操作を一つでも混ぜれば、最初の一個の故障で距離 3 の保証が崩れる。誤りを兄弟に渡さないために、論理 CNOT はブロックをまたぐ配線で組まねばならない — それが transversal の意味だ。

この「同一ブロックの二点に同時に触れない」は、初めての発想ではない。第 I 部第 4 章で、シンドローム抽出の配線を組んだとき、私たちはデータ i を専用の補助 i だけに繋ぎ、一本の補助が複数のデータに触れないようにした — 第 4 章はそれを「transversal な結線」と呼んだ。あのときは訂正装置の配線、いまは論理ゲートそのもの。場所は違っても、守っている規律は一つだ — 一個の物理操作を、同じブロックの二つの量子ビットに同時に触れさせない。第 4 章で測定を守った locality が、ここでは論理ゲートを守っている。

9.4 全量子ビットを測れば、古典デコードが論理ビットを返す

論理ゲートで計算を進めたら、最後に答えを取り出す。論理 Z の値 — 論理 0 か論理 1 か — を読む手続きを見よう。第 8 章では、誤りがないと信じてよい場面での軽い読み方を扱った。本章で問うのは、その先だ。物理量子ビットには誤りが乗っているかもしれない。誤りを抱えたまま測って、なお正しい論理ビットを読むには、どうするか。

手続きは拍子抜けするほど素朴に始まる。七個の物理量子ビットを、すべて Z 基底で測る。

各量子ビットは 0 か 1 に定まり、七桁のビット列 $m = (m_1, \dots, m_7)$ が手に入る。誤りがなければ、この m は Steane が持つ古典符号の正しい語のどれかだ。だが、どこかにビット反転 (X 誤り) が一個あれば、 m はその古典符号語が一桁化けたものになっている。

ここから先は、第7章で組んだ古典 Hamming のデコーダ、そのものだ。測った七桁 m に Hamming の検査行列 $\mathbf{H}$ を当て、シンドローム $s = \mathbf{H}m$ を mod 2 で計算する。第7章で確かめたとおり、 s は化けた桁の番地を直に指す — s を二進数として読めば、それが反転したビットの位置だ。その桁を m の上で反転して直せば、 m は正しい古典符号語に戻る。最後に、直した m の全桁の偶奇を取る。これが $Z_L = Z^{\otimes 7}$ の固有値、すなわち論理 Z の値だ (偶なら論理 0、奇なら論理 1)。

起きたことを整理する。全量子ビットを測ってビット列を得て、検査行列でシンドロームを取り、表で誤りの位置を名指して直し、それから論理パリティを読む — これは、第7章の演習で古典 Hamming に対してやった復号の手順を、測定で得たビット列にそっくり適用しただけだ。誤り込みで論理ビットを読む、という量子の手続きの正体は、測定結果に対する古典デコードなのである。量子側がやることは「全量子ビットを Z で測る」まで。あとは古典の後処理が、一個のビット反転を拾って直し、正しい論理ビットを返す。

第8章の軽い読み方との違いは、この後処理の有無にある。軽い版は、状態が綺麗な符号語だと見て論理パリティを直に読む。フル版は、同じ測定列からまずシンドロームを取り出し、一個の誤りを訂正してからパリティを読む — だから測定の直前に物理量子ビットが一個化けていても、正しい論理ビットを取り出せる。これが、誤りに耐える論理測定だ。

論理 X を読みたければ、全量子ビットを X 基底で測り、こんどは X 型生成子から成るシンドロームで位相反転 (Z 誤り) を直し、 $X_L = X^{\otimes 7}$ のパリティを読む — X と Z を入れ替えた同じ手続きになる。自己双対の符号らしく、読み出しも X - Z 対称だ。

9.5 単純さと耐性は、transversal という一つの構造の両面である

三つの観察 — 一様な H_L 、ブロック間の CNOT_L 、フル論理測定 — を、引いて眺める。前の二つは、同じ構造を共有していた。transversal、すなわち物理ゲートが一量子ビットだけに閉じるか、別ブロックの相方とだけ組むかで、同一ブロックの二量子ビットに同時には触れない、という組み方だ。そしてこの構造は、二つの仕事を同時にこなしていた。

一つは単純さだ。transversal なゲートは、ただ一様に配るか、ブロックの相方どうしを平行に繋ぐだけでかかる。同一ブロックの中で量子ビットを込み入った順に絡める振り付けは要らない。もう一つは耐性だ。その「同一ブロックの二点に触れない」localityこそが、一個の故障が同一ブロックで二個に育つ道を塞いでいた。単純さと耐性は、別々に授かった二つの幸運ではない — transversal という一つの構造を、二つの側から見たものだ。これが本章の表題、「単純さと耐性が同じ仕掛け」の意味である。

ただし、問いの初めに置いた区別を、ここで畳んでおく。耐性は locality だけで決まる。どんな transversal ゲートでも — それが何を計算するゲートであれ — 一個の故障は各ブロック高々一個の誤りに留まる。だから「transversal なゲートは、定義上 fault-tolerant であ

」と言える。これは第 I 部第 4 章の fault-tolerance の定義を、ゲートの場面へそのまま当てはめただけで、新しく証明することは何もない。一方、その単純な操作が「論理ゲート」になるかどうかは、別の条件だ。一様な $U^{\otimes n}$ が論理ゲートであるための条件は、それが S を S に写すこと、

$$U^{\otimes n} S (U^{\otimes n})^\dagger = S \iff U^{\otimes n} \in N(S)$$

—— すなわち $U^{\otimes n}$ が正規化群 $N(S)$ に属することである。観察 1 で $H^{\otimes 7}$ について確かめたのは、この一般条件の一例だった。耐性は locality がタダで配り、論理ゲート性は正規化という代数が選り分ける。この二つは別物だ。

では、Steane でこの正規化の条件を満たす transversal ゲートは何か。自己双対の CSS 符号では、アダマール H 、位相ゲート（以下サンセリフで S と書き、スタビライザ群 S と区別する）、そして CNOT が —— つまり Clifford ゲートの生成元が —— いずれも transversal にかけられ、かつ S を保つ。観察 1 の $H^{\otimes 7}$ 、観察 2 の transversal CNOT に加え、各量子ビットへ位相ゲートを一様に当てる $S^{\otimes 7}$ も論理位相ゲートとして働く。その向きは厳密化で定まる —— S は各桁で $X \mapsto Y$ 、 $Z \mapsto Z$ と作用するから、論理演算子の上では

$$S^{\otimes 7} X_L (S^{\otimes 7})^\dagger = -i X_L Z_L = -Y_L, \quad S^{\otimes 7} Z_L (S^{\otimes 7})^\dagger = Z_L$$

となる（ここで $Y_L := i X_L Z_L$ ）。これは論理 $S^\dagger$ の作用そのものであり、補正のパウリは要らない。つまり一様な $S^{\otimes 7}$ は、論理位相ゲートの逆 $S_L^\dagger$ を与える。これらを組み合わせれば、Clifford 群まるごとが transversal に手に入る。Steane が自己双対であることの利得が、ここで最大限に効いている —— 軽い生成子（第 7 章）に続いて、Clifford の全 transversal 実装が、同じ対称性から出てくる。

フル論理測定も、特定の論理パウリに限った話ではない。論理パウリを読む手続きは、その向きの全量子ビット測定が返すビット列を、古典符号でデコードすること —— シンドロームを取り、誤りを直し、論理パリティを読む —— に等しい。誤り込みの論理読み出しは、量子の測定の後ろに古典のデコーダを一つ置くこと、と一般化できる。

ここまで来ると、transversal は耐性ある論理計算の万能の道具に見えてくる。一様にかけるだけで論理ゲートになり、しかも定義上 fault-tolerant —— これで論理 Clifford がすべて揃う。だが、ちょうどここに天井がある。注意すべき誤解はこうだ —— 「transversal にかければ、どんな論理ゲートも作れる」。これは正しくない。正規化という条件は選択的で、Steane で transversal に届くのは Clifford ゲートまでだ。Clifford の外にある非 Clifford ゲート —— たとえば T ゲート —— は、 $T^{\otimes 7}$ が S を保たず、論理ゲートにならない（ T は X を X と Y の重ね合わせへ写す非 Clifford ゲートで、 $T^{\otimes 7}$ は生成子をパウリ群の外へ押し出してしまふ）。そして Clifford だけでは、量子計算は万能にならない。Clifford 回路で作れることは、古典計算機が真似できる範囲に収まってしまふからだ。万能な量子計算には、少なくとも一つ、非 Clifford のゲートが要る。

transversal は、強力だが取りこぼしのある道具だった —— 耐性ある論理 Clifford を一式タダで配る一方、その先の非 Clifford には手が届かない。この天井が、Steane に限らずある種の符号すべてに掛かる原理的なものであること（Eastin-Knill の定理）、そして天井をどう

越えるか（魔法状態）は、次章と第 III 部の主題だ。本章では、transversal が単純さと耐性を同時に生み、そのぶん Clifford で止まる — その輪郭をつかんでおけばよい。

行き先に着いた。一様な手つきが論理ゲートになるのは、それが S を保ち論理演算子を正しく動かすからで、手品ではない。誤りが増えないのは、transversal な locality が一個の故障を各ブロック一個に留めるからだ。そして単純さと耐性は、その同じ locality の両面だった。誤り込みで論理ビットを読むのは、測定列の古典デコードに過ぎない。第 6 章が符号に群を与え、第 7 章が古典符号二枚からそれを組み、第 8 章が中身を開け、本章が、その符号の上で耐性ある論理ゲートをかけ、答えを読む方法を示した。残るのは、こうして組んだ符号たちを並べて見比べ、なぜ Clifford だけでは足りないのかを正面から問うことだ。次章はそこへ向かう。

9.6 演習

本章の演習は、「transversal が踏みとどまるところ」と「非 transversal が壊れるところ」を手元で見比べ、誤り込みの論理読み出しを実装するためにある。解答は付録 C にまとめる。

演習 9-1. transversal CNOT が誤りを兄弟に渡さないことを確かめる。二つの符号ブロック（各七量子ビット）の間に、量子ビット i と量子ビット i を結ぶ transversal CNOT を NumPy で組め。制御ブロックの一つの量子ビットに X 誤りを一個置き、CNOT を通した後の誤りの台 (support) を調べる。誤りが、標的ブロックの対応する一個の量子ビットにだけ移り、どちらのブロックでも兄弟（他の量子ビット）には乗らないこと、すなわち各ブロックが weight 1 に留まることを確認する。 Z 誤りを標的側に置いた場合も、制御側の対応する一個へ移ることを見るとよい。これが観察 2 のトレースの、数値での裏取りだ。

演習 9-2. 非 transversal な操作が一個の誤りを二個に育てることを確かめる。今度は同じブロックの中の二つの量子ビット（量子ビット 3 を制御、量子ビット 5 を標的など）に物理 CNOT を当てる。制御側の量子ビット 3 に X 誤りを一個置いて通すと、同じブロックの量子ビット 5 にも X が移り、ブロックが weight 2 を抱えることを確認せよ。さらに、この weight 2 の誤りに対してシンδροームを計算し、それが単一誤りの訂正表では正しく直せない — 別の単一誤り、または論理演算子と区別がつかない — ことを見る。transversal を捨てると最初の一個の故障で距離 3 の保証が崩れる、という観察 2 の対比を、自分の手で壊して確かめるのが狙いだ。

演習 9-3. 誤り込みの論理測定を実装し、論理ベル対の相関を読む。論理状態を全量子ビット Z 基底で測り、得たビット列に Hamming のシンδροームを当て、誤り位置を表で引いて訂正し、全桁の偶奇から論理 Z を読む手続きをコードに起こせ（ X 基底側も同様に論理 X を読む）。仕上げるに、二つの論理量子ビットで論理ベル対を作り、理想実行のもとで $Z_{L,A}Z_{L,B} = X_{L,A}X_{L,B} = +1$ という基本相関が出ることを確認する。誤り込みの論理読み出しが古典デコードそのものであることを、測定列に対する後処理として体感するのが目的だ。（第 I 部・他章の Notebook とは独立に、本章だけで閉じる実装でよい。）

（残留誤りの分布・訂正可否の具体や、論理ベル対の測定統計は、付録 C の解答で数値とともに

に与える。)

第 10 章

符号の比較とスタビライザ符号の限界 —— なぜ Clifford だけでは足りないか

10.1 Steane を選んだ理由は、並べて初めて見える

第 9 章は、組み上げた符号を並べて見比べ、なぜ Clifford だけでは足りないのかを正面から問う、という課題を残して閉じた。本章はこれを引き取る。

第 II 部で実際に構成した符号は、ひとつである。第 6 章でスタビライザ群の言葉を整え、第 7 章で古典 Hamming 符号二枚から Steane を構成し、第 8 章でその論理状態を書き下し、第 9 章で Steane 上の耐性ある論理 Clifford と論理測定を与えた。四つの章は、いずれも Steane を対象としていた。しかし、その選択そのものは問われていない —— なぜ Steane なのか。

本書が最初に扱った符号は、Steane ではない。第 3 章から第 6 章まで一貫して例に用いたのは Shor 符号 —— 物理量子ビット 9 個に 8 本の生成子を課した $[[9, 1, 3]]$ —— であった。第 7 章で対象は 7 量子ビットの Steane $[[7, 1, 3]]$ へ移り、以降の構成はすべてその上に積まれている。移行には理由があるはずだが、距離だけを基準にとる限り、両者は区別されない。どちらも論理 1 量子ビットを距離 3、すなわち単一誤りを訂正できる強さで保護する。第 I 部以降、符号の強さは距離で測られてきた。その基準の上では、Shor と Steane は同位である。同位の符号の一方を選んだ根拠は、距離の外にある。

基準が一つでは不足する、ということである。第 7 章で weight —— 生成子が恒等でなく作用する桁の数 —— を、距離とは独立な第二の基準として導入し、Steane の生成子が weight 4、Shor の位相型生成子が weight 6 であることを確認した。距離 3 で並ぶ符号も、weight では分かれる。基準は weight に限らない。シンジロームを余さず用いているか、論理ゲートをどの程度直接に実装できるか —— 距離が捨象していたこれらの差が、符号選択の分岐点となる。本章はまず、距離 3 の符号を三つ並べ、距離の測らない量で比較する。

比較は「なぜ Steane か」に解答を与える。だが同じ場面で、より大きな問いが現れる。第 9 章では、Steane 上の論理 Clifford —— アダマール・位相・CNOT —— が、いずれも一様な

transversal 操作として、耐性を保ったまま実装できることを見た。耐性ある論理ゲートがこれだけ容易に得られるならば、transversal の積み重ねによって耐性ある万能量子計算が構成できるように思われる。しかし、そうはならない。第 9 章末で予告したとおり、transversal で到達できるのは Clifford までであり、その先には天井がある。耐性ある論理ゲートを transversal で全て揃えてもなお、万能計算には届かない。その理由を問う。

本章の到達点を先に示す。第一に、距離 3 の三符号 — Shor $[[9, 1, 3]]$ ・五量子ビット $[[5, 1, 3]]$ ・Steane $[[7, 1, 3]]$ — を、weight・シンドロームの利用効率・transversal で到達できるゲートの範囲で比較すると、Steane が論理計算に最も適した符号として現れる。その過程で、シンドロームを一切無駄にしない「完全」符号である五量子ビットが、その効率と引き換えに何を手放しているかも示す。第二に、三符号には共通の天井が存在する — スタビライザ符号で構成できる範囲は Clifford に閉じ、Clifford で実現できる計算は古典計算機で模倣できる (Gottesman–Knill)。さらに、transversal が Clifford を超えられないことは Steane に固有の事情ではなく、原理的な制約である (Eastin–Knill)。第三に、この天井を越えるには Clifford の外の非 Clifford ゲート — 標準的には T ゲート — が必要であり、それを耐性を保って供給する方法、すなわち魔法状態が第 III 部の主題となる。距離の比較から出発した本章は、第 II 部全体を、この単一の天井の手前で閉じる。

10.2 距離が同じ三符号は、weight と構造で分かれる

比較の卓に載せる符号を確定する。一つは Shor $[[9, 1, 3]]$ 、第 3 章以来の例である。一つは Steane $[[7, 1, 3]]$ 、第 7 章で構成し、以降の土台とした符号である。これに、距離 3 の符号をもう一つ加える — 五量子ビット符号 $[[5, 1, 3]]$ である。物理量子ビットを 5 個しか用いない、単一誤りを訂正できる最小の符号だ。その生成子は、各桁に $X \cdot Z$ ・恒等を配した weight 4 の演算子 4 本からなる（具体形は本章では立ち入らない）。Shor や Steane と異なり、その生成子は X 型と Z 型に分かれない — すなわち CSS ではない。

三つを、距離以外の量で並べる（表 10.1）。

表 10.1: 距離 3 の三符号を、距離以外の量で並べる。距離 3・論理 1 量子ビットはすべて共通だが、CSS 性・生成子の weight・transversal で届く Clifford の範囲は揃わない。

符号	$[[n, k, d]]$	CSS	生成子の weight	transversal で届く Clifford	提案 (年・人)
Shor	$[[9, 1, 3]]$	CSS (非自己双対)	Z 型 2 / X 型 6	CNOT は届く・ H は届かない	1995, Shor
五量子ビット	$[[5, 1, 3]]$	非 CSS	4 (一様)	一式は届かない	1996, Laflamme ら
Steane	$[[7, 1, 3]]$	CSS (自己双対)	4 (一様)	$H \cdot S \cdot \text{CNOT}$ すべて届く	1996, Steane

距離はすべて 3、論理量子ビットはすべて 1。だが、その他の欄はそろっていない。
 ここで一言、距離が約束するものの範囲を確かめておく。距離 3 という共通の値が保証するのは、あくまで単一誤りの訂正までである — これを第 9 章の演習 9-2 で、同一ブロックに

weight 2 の X 誤りが乗ると、そのシンδροームが別の単一誤りと別名衝突を起こし、訂正しようとするばかえって論理 X になってしまう実例として、具体に見た。距離は符号の強さの一つの目安だが、それ自体が符号の強さのすべてではない。以下で比較する weight や構造は、距離が同じ符号の優劣を分ける、距離の外の量である。

物理量子ビットの数では、五量子ビットが最小で、Shor が最大だ。誤りを訂正するのに要する物理量子ビットだけを見れば、五量子ビットが最も儉約している。

生成子の weight では、Steane と五量子ビットが 4 でそろい、Shor だけが —— 位相型生成子で —— 6 と重い。第 7 章で見たとおり、Shor の位相検査は三つのブロックをまたいで 6 桁に触れねばならず、それが weight を押し上げる。weight が後で（観察 3）しきい値に効くことを思えば、この差は小さくない。

そして、論理計算のしやすさを最も分けるのが、transversal で届く Clifford ゲートの広さだ。第 9 章で、自己双対の CSS 符号では $H \cdot$ 位相 $\cdot$ CNOT のすべてが transversal にかかるとを見た。Steane はまさにその符号で、Clifford 群がまるごと一様な手つきで手に入る。Shor も CSS だが自己双対ではない —— X を直す側（繰り返し符号）と Z を直す側（ブロックまたぎ）が同形でない —— から、CNOT は transversal に届くが、 H は届かない。五量子ビットは CSS ですらないため、自己双対の議論が立たず、Clifford を一式 transversal に揃えることができない。

ここに、なぜ Steane か、の最初の解答がある。五量子ビットは最小だが、論理ゲートを transversal で揃える点で劣る。Shor は CSS だが重く、しかも H が transversal に届かない。Steane だけが、軽い生成子と、Clifford の全 transversal 実装を同時に持つ —— どちらも自己双対という一つの対称から出た性質だ（第 7 章・第 9 章）。論理計算を耐性を保って積み上げる土台として、Steane が最も据わりがよい。

ただし、五量子ビットを「劣る」とだけ片づけるのは早い。距離以外の欄には、まだ見えない美点がある。次節で、その符号が何に秀でているかを見る。

10.3 五量子ビットの「完全」は、CSS の対称と引き換えである

五量子ビットが秀でているのは、シンδροームの使い方にある。第 6 章で見たとおり、生成子を s 本立てれば、状態の空間は 2^s 個の部分空間に分かれ、シンδροームの組はそのどれにいるかを指す住所だった。五量子ビットは $n = 5$ 、論理 1 量子ビットだから生成子は 4 本、シンδροームは $2^4 = 16$ 通りある。

この 16 通りが、訂正すべき誤りの数とちょうど一致する。距離 3 の符号が直すべきは weight 1 までの誤り —— すなわち、どこにも誤りのない場合（1 通り）と、5 個の量子ビットのいずれか一つに $X \cdot Y \cdot Z$ のいずれかが乗った場合（ $3 \times 5 = 15$ 通り） —— で、合わせて $1 + 15 = 16$ 通りである。16 個のシンδροームが、16 個の訂正対象に、余すところなく一対一で対応する。シンδροームに一つの無駄もない。

定義 10.1 (完全符号) 訂正すべき誤りの総数が、シンδροームの総数 2^{n-k} をちょうど使い切る符号を**完全符号** (perfect code) と呼ぶ。距離 3・論理 $k = 1$ の場合、訂正対象は weight

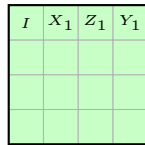
≤ 1 の誤り $1 + 3n$ 個であり、完全性の条件は

$$1 + 3n = 2^{n-k}$$

と書ける。一般に訂正対象の総数はシンドローム数を超えられないから（球充填限界 $1 + 3n \leq 2^{n-k}$ ）、完全符号はこの限界に等号で達する符号にほかならない。誤りの及ぶ「球」が符号空間のまわりを隙間なく敷き詰める、という幾何の言葉でも言い表される性質である。

五量子ビットは $1 + 3 \times 5 = 16 = 2^4$ で、この等号を満たす — 距離 3・論理 1 量子ビットを保つのに要する物理量子ビットの下限に達した、最も儉約な単一誤り訂正符号である。最小であることと、シンドロームを使い切ることとは、同じ完全性の二つの顔だ。

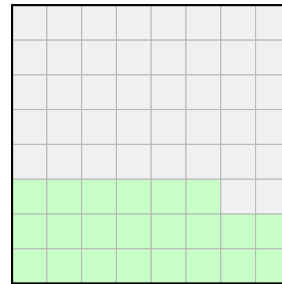
五量子ビット（完全）



$2^4 = 16$ シンドローム = weight ≤ 1 の 16 誤り。

隙間も重なりもなく敷き詰まる。

Steane（非完全）



$2^6 = 64$ シンドロームのうち 22 が誤りに対応（緑）、残り 42 は空く（灰）。

図 10.1: 完全符号では weight ≤ 1 の誤りの球がシンドローム空間を隙間なく敷き詰める。五量子ビットは飽和（完全）、Steane は余を残す（非完全）。

Steane は、この意味では完全ではない。生成子は 6 本、シンドロームは $2^6 = 64$ 通りあるが、weight 1 までの訂正対象は $1 + 3 \times 7 = 22$ 通りしかない。差し引き 42 通りのシンドロームが、単一誤りの訂正には使われずに残る。だが、この「余り」は無駄ではない。Steane の X 型生成子は位相反転（ Z 誤り）を、 Z 型生成子はビット反転（ X 誤り）を、別々に検出していた。シンドロームが多いのは、 X の誤りと Z の誤りを独立に、それぞれの位置まで読み分けているからだ。五量子ビットがシンドロームを使い切るのは、 X と Z をまとめて一括に扱う（CSS でない）からであり、Steane がシンドロームを余らせるのは、 X と Z を分けて扱う（CSS である）からだ。

ここで言葉を分けておく。「完全」は、シンドロームを使い切る符号（五量子ビット）の意味に使う。Steane が持つのは別の性質 — X と Z を独立に検出すること — であって、これを「完全」とは呼ばない。同じ符号の良し悪しでも、両者は異なる美点である。

そして、五量子ビットの完全性には代償がある。シンドロームを使い切るその構造は、 X と Z を一括に扱うこと、すなわち CSS でないことと表裏だった。CSS でない以上、自己双対という対称も立たず、観察 1 で見たとおり Clifford を一式 transversal に揃えることができない。五量子ビットは、単一誤りを直すだけなら最も儉約な符号でありながら、その儉約と引き換えに、論理計算を transversal で素直に積み上げる足場を手放している。儉約に最も秀でた

符号が、計算には最も不向きだ —— 距離だけを見ては決して見えなかった、符号選択の機微である。

10.4 軽い生成子は、しきい値を押し上げる

第7章で weight を導入したとき、一つの約束を後の章に預けた —— 生成子が軽いほど、それを測る補助の仕掛けが小さくて済み、測定そのものが持ち込む誤りの種が減る。だから weight が小さいほど、誤りに耐える敷居が上がる、と。その敷居 —— しきい値 —— は、第I部第5章で正確な形を得ている。ここで、weight としきい値を、半定量につなぐ。

生成子を測るとき、ただ測るだけでは足りない。測定に用いる補助の量子ビット自身が故障すると、その故障が、測られているデータ側に複数の誤りとなって流れ込むことがある。これを防ぐ —— 一つの故障を、データの上で高々一つの誤りに留める —— には、補助の側を、生成子の桁数に応じた仕掛けに組まねばならない。weight 4 の生成子を測る仕掛けは、weight 6 の生成子を測る仕掛けより小さくて済む。触れる桁が少ないぶん、用意する補助量子ビットも、通すゲートも少ないからだ。

仕掛けが小さいことが、なぜしきい値を上げるのか。第5章を思い出す。接続でしきい値が決まる最も単純なモデルでは、しきい値は $p_{th} = 1/A$ であった。ここで A は、論理誤りを引き起こす「悪性の故障対」の数で、故障が起こりうる箇所 —— ゲート・測定・待機のステップ —— の数 N に対して、およそ N から二つ選ぶ組合せの数 $\binom{N}{2}$ だった。故障箇所が増えれば A は増え、しきい値 $1/A$ は下がる。

つなぐと、こうなる。Shor の weight 6 の生成子を耐性を保って測るには、Steane の weight 4 の生成子を測るより大きな仕掛けが要る。仕掛けが大きいほど故障箇所 N が増え、 N が増えれば悪性の対 A が増え、 A が増えればしきい値 $1/A$ が下がる。逆に、Steane の軽い生成子は、測定の仕掛けを小さく保ち、故障箇所を減らし、しきい値を高く保つ。軽い生成子は、しきい値を押し上げるのである。

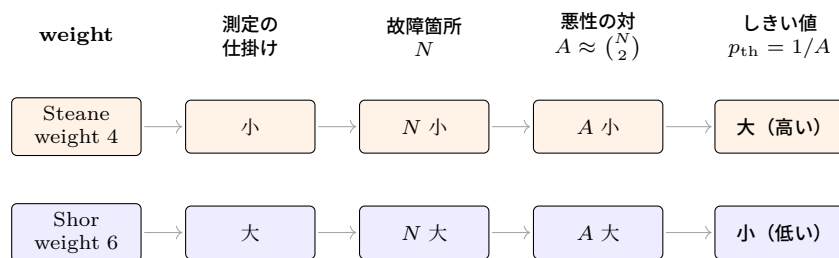


図 10.2: 軽い生成子 → 小さい測定回路 → 少ない故障箇所 → 高いしきい値、という半定量の連鎖。

ただし、この接続は半定量にとどまる。しきい値を決める A は、シンδροーム測定の仕掛けだけでなく、訂正回路の全体で決まる量であり、ここでは「生成子が軽いほど測定の故障箇所が減る」という一筋だけを取り出して A の増減に結びつけた。正確なしきい値は、用いる手順の全体を数えて初めて定まる。それでも、向き —— 軽い生成子がしきい値を上げる —— は確かであり、これが、Steane のような低 weight の符号が好まれる、距離とは独立なもう一

つの理由になっている。

これで、なぜ Steane か、の解答がそろった。Steane の自己双対は、二つの利点を同じ源から与えていた — Clifford の全 transversal 実装（観察 1）と、低い weight ゆえの高いしきい値（観察 3）である。論理計算を耐性を保って積み上げる土台として、Steane が最も据わりがよい理由が、これで言葉になった。

10.5 どの符号も、transversal だけでは Clifford の天井を越えない

三つの符号を並べ、Steane が論理計算の土台として最も据わりがよいと見た。ここで、視点をもう一段引く。Steane の上では Clifford がまるごと transversal に手に入る — では、その Clifford で、いったいどこまで計算できるのか。そして、この問いの答えは、Steane だけのものなのか、それとも符号一般に共通の何かなのか。

まず、Clifford がどこまで届くかを問う。第 9 章の終わりに、Clifford でできることは古典計算機が真似できる範囲に収まる、と述べた。これには名前がある。

定理 10.1 (Gottesman–Knill) スタビライザ状態 — $|0 \cdots 0\rangle$ から Clifford 回路で到達でき、Pauli 測定を施した状態 — と、その上の Clifford 演算は、古典計算機で効率よくシミュレートできる。

直感はこうだ。スタビライザ状態は、それを固定する生成子（ n 本の Pauli 演算子）で完全に決まり、Clifford ゲートはその生成子を単純な規則で別の Pauli へ移す。だから、状態を 2^n 次元のベクトルで持つ必要はなく、 n 本の生成子（物理量子ビット数に比例する量）を追うだけで状態の変化を完全に追える。指数的なベクトルが要らないなら、古典計算機が追いつく。帰結は重い — たとえある符号が Clifford 群をまるごと transversal に与えても、Clifford だけの計算は、古典計算機に対する優位を生まない。観察してきた transversal Clifford の豊かさは、それ自体では、量子計算の旨味に届かないのである。（定理の証明は原論文に譲る。）

優位を得るには、Clifford の外へ出るほかない。では、Clifford の外の非 Clifford ゲートを、どれか別の符号でなら transversal に手に入れられるのか。ここで二つ目の定理が、その道を塞ぐ。

定理 10.2 (Eastin–Knill) 誤りを検出できる（自明でない距離を持つ）符号では、transversal にかかけられる論理ゲートの全体は、有限個からなる群にとどまる。

万能な量子計算には、連続的に — いくらでも細かい角度まで — ゲートを刻めることが要るが、有限個の群はそれを与えられない。だから、どんな符号をもってしても、transversal なゲートだけで万能集合を揃えることはできない。第 9 章で見た「Steane の transversal は Clifford で止まる」という天井は、Steane の事情ではなく、この一般法則の一つの現れだったのだ。符号を取り替えても、この天井は越えられない。（定理の厳密な前提と証明は原論文に譲る。）

二つの定理が、挟み撃ちにする。Clifford だけでは古典に勝てない (Gottesman–Knill)。かといって、transversal だけでは Clifford の外 — 非 Clifford — に出られず、有限群を超えられない (Eastin–Knill)。耐性を保ったまま万能に計算するには、少なくとも一つ、非 Clifford のゲート — 標準的には T ゲート — を加えねばならず、しかもそれは transversal の手つきだけからは得られない。

ならば、その非 Clifford ゲートを、どうやって耐性を保って供給するのか。道は、ゲートを直に transversal にかけることを諦め、代わりに特別な状態 — 魔法状態 (magic state) — をあらかじめ用意しておき、それを消費して T ゲートの作用を得る、という迂回にある。その魔法状態をどう耐性を保って準備するか、その手順 (蒸留) と構成こそが、第 III 部の主題になる。本章は、その入口 — 天井を越える鍵が魔法状態にあること — を示すにとどめ、中身は第 III 部に渡す。

第 II 部を、ここで閉じる。第 6 章はスタビライザ群という言葉で符号に代数を与え、第 7 章は古典符号二枚から CSS・Steane を構成し、第 8 章はその論理状態を開け、第 9 章は耐性ある論理 Clifford と論理測定を与えた。そして本章は、組んだ符号を並べて見比べ、なぜ Steane が選ばれるかに答え、同時に、どの符号にも共通する Clifford の天井を見た。「設計する」の数学は、この一枚の天井の前で一区切りとなる。次の第 III 部「スケールさせる」の数学は、表面符号とデコーダ、そして本章が入口だけを示した魔法状態を扱い、この天井の向こうへ進む。

10.6 演習

本章の演習は、三符号の比較を自分の手で確かめ、Gottesman–Knill の天井を体感し、五量子ビットが何を手放しているかを具体に見るためにある。解答は付録 C にまとめる。

演習 10-1. 三符号の比較表を、自分で埋め直す。Shor $[[9, 1, 3]]$ ・五量子ビット $[[5, 1, 3]]$ ・Steane $[[7, 1, 3]]$ について、物理量子ビット数・距離・CSS か否か・生成子の weight・transversal で届く Clifford ゲートを表にまとめよ。そのうえで、「論理計算を耐性を保って積み上げる土台として、なぜ Steane を選ぶか」を、自分の言葉で二つの軸 — 自己双対ゆえの Clifford 全 transversal (観察 1) と、低い weight ゆえの高いしきい値 (観察 3) — に触れて述べよ。紙の上の演習である。

演習 10-2. スタビライザ状態が古典計算機で追えることを確かめる。まず、Clifford 回路 (たとえば H と CNOT だけからなる、論理状態を作る回路) を状態ベクトルで実行し、得た状態を固定するスタビライザ生成子 n 本を読み取る NumPy を書け。次に、状態ベクトルを持たずに、その n 本の生成子だけを Clifford ゲートの共役規則 (各 Pauli がどの Pauli に移るか) で更新する簡易版を実装し、両者が同じ状態を表すことを確認する。生成子を追うだけで — 指数的なベクトルなしに — Clifford 回路が追えること、これが Gottesman–Knill の体感だ。**動かして壊す:** この生成子の更新規則に T ゲートを一つ混ぜてみよ。 T は生成子の Pauli を Pauli に移さないため、簡易版はもう状態を追えなくなる — Clifford の外に出た瞬間に、スタビライザによる記述が破れることを、手元で確かめる。

演習 10-3. [発展] 五量子ビットが CSS でないこと、そして Clifford を一式 transversal

に持たないことを調べる。五量子ビットの 4 本の生成子は、 $XZZXI$ とその巡回シフト ($IXZZX$, $XIXZZ$, $ZXIXZ$) で与えられる。各生成子が同じ桁に X と Z の両方を含み、 X 型・ Z 型に分けられないこと (非 CSS) を確認せよ。さらに、各量子ビットに H を一様にかける $H^{\otimes 5}$ が、これらの生成子の集合を自分自身に戻さないこと — すなわち transversal H が論理ゲートにならないこと — を、生成子を H で共役して調べよ。Steane では $H^{\otimes 7}$ が生成子の集合を保った (第 9 章) のと対照的である。

(残差誤りの分布や訂正可否、簡易タブローと状態ベクトルの一致の精密な検証は、付録 C の解答で数値とともに与える。本散文では数値を断定しない。第 I 部・他章の Notebook とは独立に、本章だけで閉じる実装でよい。)

付録 A

記法・記号一覧

本付録は、第 II 部の各章および付録 B・C で用いる記号を一箇所にまとめる。掲載した記号は本文での実際の用法から起こしたものであり、第 I 部には独立した記法表があるため（本書は分冊運用）、ここでは第 II 部に現れる記号に絞る。成分を $0, 1$ に限り和を $\text{mod } 2$ で取る算術を、本書では二元体 $\text{GF}(2)$ （その集合表記 $\mathbb{F}_2 = \{0, 1\}$ ）と呼ぶ。詳しい $\text{GF}(2)$ 上の線形代数は付録 B にまとめてある。

A.1 体・ビット演算と量子状態

記号	意味
$\mathbb{F}_2 = \{0, 1\}$	二元体 $\text{GF}(2)$ 。成分は $0, 1$ 、和は $\text{mod } 2$
$\oplus$	$\text{GF}(2)$ の加法（ビット単位の排他的論理和、 $1 \oplus 1 = 0$ ）
$\text{mod } 2$	2 を法とする合同
$ \psi\rangle$	量子状態（ケット）
$\langle\psi $	その双対ベクトル（ブラ）
$\langle\phi \psi\rangle$	内積。 $\langle 0_L 1_L\rangle = 0$ は論理基底の直交性
$ 0\rangle, 1\rangle$	1 量子ビットの計算基底
$ +\rangle, -\rangle$	アダマール基底、 $ \pm\rangle = (0\rangle \pm 1\rangle)/\sqrt{2}$
$ 0_L\rangle, 1_L\rangle$	論理基底状態（符号空間内の論理 0・論理 1）
$ +_L\rangle, -_L\rangle$	論理アダマール基底、 $ \pm_L\rangle = (0_L\rangle \pm 1_L\rangle)/\sqrt{2}$
$\otimes$	テンソル積。 $X^{\otimes 7}$ は X を 7 個並べた演算子

A.2 演算子とゲート

記号	意味
X, Y, Z	パウリ演算子。 X はビット反転、 Z は位相反転、 $Y = iXZ$
$\mathbb{1}$	恒等演算子 ($\mathbb{1}$ 、dsfont)
H	アダマールゲート。検査行列 $\mathbf{H}$ と区別するためサンセリフで表す
S	位相ゲート。スタビライザ群 S (数式斜体) と区別するためサンセリフで表す
T	T ゲート ($\pi/8$ 回転)。Clifford の外にある非 transversal な論理ゲート
X_L, Z_L	論理パウリ演算子。Steane では $X_L = X^{\otimes 7}$, $Z_L = Z^{\otimes 7}$
$[A, B]$	交換子。 $[A, B] = 0$ は可換 ($[g, L] = 0$ が測定保存の核心)
tr	トレース

注意 A.1 ゲートのうち H と S のみサンセリフ字体を用いる。これは検査行列 $\mathbf{H}$ ・スタビライザ群 S と同じ字母が衝突するためで、衝突しない X, Z, T などは通常の数式字体のままとする。

A.3 スタビライザ符号と符号パラメータ

記号	意味
$S = \langle g_1, \dots, g_s \rangle$	スタビライザ群。 $\langle \cdot \rangle$ は内側の元が生成する群
g_i	スタビライザ群 S を生成する独立な演算子 (生成子)
$\mathcal{P}$	パウリ群
$[[n, k, d]]$	n 物理量子ビット・ k 論理量子ビット・距離 d の量子符号
d	符号距離 (確実に訂正できるのは $\text{weight} \leq \lfloor (d-1)/2 \rfloor$)
weight	パウリ演算子の weight (恒等でない桁の数)
シンδροーム	全生成子の測定が返す ± 1 の組。誤りの行き先を名指す
符号空間	すべての生成子が $+1$ を返す部分空間 (符号語の住まい)

A.4 古典線形符号と GF(2) 線形代数

CSS 構成と検査行列の扱い (第 7 章・付録 B) で用いる。

記号	意味
C	古典線形符号 ($\text{GF}(2)$ 上の部分空間)
$C^\perp$	双対符号。 $C^\perp \subseteq C$ を双対包含と呼ぶ
$C = C^\perp$	自己双対 (Steane が満たし、CSS の対称の源)
$\mathbf{H}$	検査行列 (parity-check matrix)。太字で表す
$\mathbf{G}$	生成行列
$\mathbf{H}^\top$	転置
$\mathbf{H}\mathbf{H}^\top = 0$	双対包含の判定式 (mod 2)
$\mathbf{H}v = 0$	符号語の条件 (v は符号語)
$\text{rank } \mathbf{H}$	検査行列の階数
$\ker \mathbf{H}$	核 ($\mathbf{H}v = 0$ の解空間)
$\dim$	次元

A.5 しきい値（半定量）

第 10 章の weight としきい値の半定量的な接続で用いる（正確なしきい値は手順全体で決まる）。

記号	意味
p_{th}	しきい値。接続の最も単純なモデルで $p_{\text{th}} = 1/A$
A	論理誤りを生む「悪性の故障対」の数、 $A \approx \binom{N}{2}$
N	故障が起こりうる箇所（ゲート・測定・待機）の数
$\binom{N}{2}$	N から 2 つ選ぶ組合せの数

付録 B

GF(2) 上の線形代数

本付録は、第 II 部の各章で繰り返し用いる GF(2) (二元体) 上の線形代数を一箇所にまとめる。検査行列の階数から符号の次元を読むこと、双対符号が検査行列の行空間として現れること、双対包含 $C^\perp \subseteq C$ を $\mathbf{H}\mathbf{H}^\top = 0 \pmod{2}$ で判定すること。これらはすべて、成分を 0, 1 に制限し、すべての算術を mod 2 で行う線形代数である。実数体・複素数体の線形代数とほとんど同じ手順で進むが、いくつかの直感は GF(2) で破れ、その破れこそが量子符号の構成で効く。数値はすべて NumPy による独立計算で確認した。

B.1 二元体 $\mathbb{F}_2$ とベクトル空間

第 II 部で扱うビット列・検査行列は、すべて「0 と 1 だけ」を成分にもち、和を「mod 2」で取った。この算術は、ちょうど 2 つの元からなる体 GF(2) の算術である。本書では二元体を GF(2) と書き、その集合としての記号を $\mathbb{F}_2 = \{0, 1\}$ とする。

定義 B.1 (二元体 GF(2)) 集合 $\mathbb{F}_2 = \{0, 1\}$ に、次の和と積を入れたものを二元体 GF(2) と呼ぶ。

+	0	1	×	0	1
0	0	1	0	0	0
1	1	0	1	0	1

和は通常の整数の和を 2 で割った余り、積も同様である。

GF(2) には、実数体や複素数体には無い便利な性質がいくつかある。これらは第 II 部の計算からマイナス記号を消し、すべてを「足すか足さないか」に還元する。

- $1 + 1 = 0$ 。したがって $-1 = 1$ であり、**減法は加法に一致する** ($a - b = a + b$)。GF(2) 上の計算にマイナス記号は現れない。
- 0 でない元はただ一つ (1) で、それは自分自身の乗法逆元である ($1 \times 1 = 1$)。割り算は 1 で割ることに尽き、実質不要。
- 任意の元 a について $a + a = 0$ 。

長さ n のビット列全体を、この体の上のベクトル空間とみなす。

定義 B.2 (ベクトル空間 $\mathbb{F}_2^n$) $\mathbb{F}_2^n = \{0, 1\}^n$ を、成分ごとの mod 2 和と、スカラー (0 または 1) 倍を演算とするベクトル空間とする。零ベクトルは全成分 0、各ベクトル v の加法逆元は v 自身 ($v + v = 0$) である。

実数体上のベクトル空間と同じく、一次結合・一次独立・基底・次元がそのまま定義できる。違いはスカラーが $\{0, 1\}$ に限られる点だけである。とくに $\mathbb{F}_2^n$ の元はちょうど 2^n 個ある。

例 B.1 ($\mathbb{F}_2^3$) $\mathbb{F}_2^3$ は $2^3 = 8$ 個のベクトル 000, 001, 010, 011, 100, 101, 110, 111 からなる。和は桁ごとの排他的論理和に等しい。たとえば

$$110 + 011 = 101, \quad 101 + 101 = 000.$$

後者は $v + v = 0$ の一例である。任意の二つのベクトルの和がふたたび $\mathbb{F}_2^3$ に属すること (加法閉性)、すべての v について $v + v = 0$ となることは、8 個の元すべてについて直接確かめられる。

B.2 $\mathbb{F}_2$ 上の行列とガウス消去・階数

GF(2) 上でも、連立一次方程式を解いたり行列の階数を求めたりする手順は、実数体上の線形代数とほぼ同じである。ただ一つの規則は「すべての算術を mod 2 で行う」こと。むしろ手順は簡単になる。スカラーが 0, 1 しかないため、行のスカラー倍 (正規化) が不要で、行基本変形は次の二つに尽きる。

- 二つの行を入れ替える。
- ある行に別の行を足す (mod 2)。

これらを使って、各列で先頭の 1 (枢軸, pivot) を立て、同じ列の他の 1 を消していく。掃き出しが終わったとき、枢軸をもつ行の本数が行列の階数である。

定義 B.3 (GF(2) 上の階数) 成分が $\mathbb{F}_2$ の行列 M の階数 $\text{rank } M$ を、 M を行基本変形 (mod 2) で簡約したときの枢軸の個数、すなわち一次独立な行の最大本数と定める。

ここで、実数体に慣れた直感が一つ破れる。**階数は、係数をどの体で考えるかに依存する。** 同じ整数行列でも、実数体上の階数と GF(2) 上の階数は一致しないことがある。

例 B.2 (体によって階数が変わる)

$$A = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}.$$

GF(2) 上では、第 3 行が第 1 行と第 2 行の和に等しい ($1100 + 0110 = 1010, \text{ mod } 2$)。したがって 3 本の行は一次従属で、 $\text{rank}_{\mathbb{F}_2} A = 2$ である。一方、実数体上では同じ第 3 行は第 1・第 2 行の (実数係数の) 一次結合では書けず、 $\text{rank}_{\mathbb{R}} A = 3$ となる。

この差は実害になる。GF(2) 上の階数を、NumPy の `numpy.linalg.matrix_rank` で求めてはいけない。この関数は浮動小数点（実数体）で階数を計算するため、上の A には 3 を返す。GF(2) では mod 2 の掃き出しを自前で行う必要がある。次のコードは、mod 2 のガウス消去で階数を求める関数と、両者が食い違う様子を示す。

```

1  import numpy as np
2
3  def gf2_rank(M):
4      """Rank of a 0/1 matrix over GF(2) (Gaussian elimination mod 2).
5          """
6      M = (np.array(M, dtype=int) % 2).copy()
7      r = 0
8      rows, cols = M.shape
9      for c in range(cols):
10         piv = next((i for i in range(r, rows) if M[i, c]), None)
11         if piv is None:
12             continue
13         M[[r, piv]] = M[[piv, r]]          # swap pivot row up
14         for i in range(rows):             # add pivot row to clear
15             if i != r and M[i, c]:        column c
16                 M[i] = (M[i] + M[r]) % 2
17         r += 1
18     return r
19
20 A = np.array([[1, 1, 0, 0],
21               [0, 1, 1, 0],
22               [1, 0, 1, 0]])
23 print("rank over GF(2) :", gf2_rank(A))          # -> 2
24 print("matrix_rank (R) :", int(np.linalg.matrix_rank(A.astype(float)
25 )))      # -> 3

```

rank over GF(2) : 2

matrix_rank (R) : 3

本付録を通じての走行例として、第 7 章以降で使った Steane 符号の（古典 Hamming）検査行列を採る。列を 1 から 7 までの二進表現とした

$$\mathbf{H} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

は 3×7 行列で、その GF(2) 上の階数は 3 である（3 本の行は一次独立）。この $\text{rank } \mathbf{H} = 3$ が、 $\mathbf{H}$ の定める符号の次元 ($k = 7 - 3 = 4$) と双対符号の次元 (3) の両方を決める。次節 (B.3) で行空間・核と次元の関係を、B.4 でそれを符号の言葉に翻訳する。

B.3 行空間・核と次元

検査行列 $\mathbf{H}$ は、 $\mathbb{F}_2^n$ の中に二つの部分空間を定める。行が張る行空間と、 $\mathbf{H}$ を掛けて 0 になるベクトルの集合である核 (kernel, 零空間) である。符号の構成では、この二つがそれぞれ別の役割を担う。

定義 B.4 (行空間と核) 成分が $\mathbb{F}_2$ の $m \times n$ 行列 M について、

$$\text{行空間} = \{ aM : a \in \mathbb{F}_2^m \}, \quad \ker M = \{ x \in \mathbb{F}_2^n : Mx^\top = 0 \}$$

と定める。行空間は M の行の一次結合全体、核は M を左から掛けて消えるベクトル全体である。

行空間の次元は、定義より一次独立な行の本数、すなわち階数 $\text{rank } M$ に等しい。核の次元は、次の定理が与える。

命題 B.1 (階数・退化次数定理 (GF(2))) 成分が $\mathbb{F}_2$ の $m \times n$ 行列 M について

$$\dim(\ker M) + \text{rank } M = n.$$

証明. M を行基本変形 (mod 2) で簡約すると、枢軸をもつ列が $\text{rank } M$ 本、枢軸をもたない自由列が $n - \text{rank } M$ 本に分かれる。各自由変数に独立に 1 を立て、残りの枢軸変数を方程式 $Mx^\top = 0$ から一意に定める割り当ては、核の一次独立な基底を与える。その本数は自由列の数 $n - \text{rank } M$ に等しいから、 $\dim(\ker M) = n - \text{rank } M$ である。掃き出しの算術が mod 2 になるだけで、議論は実数体の場合と同一である。 $\square$

部分空間 $V \subseteq \mathbb{F}_2^n$ の元はちょうど $2^{\dim V}$ 個あるので、次元は要素数を数えれば読める。走行列の $\mathbf{H}$ (3×7 , $\text{rank } \mathbf{H} = 3$) では、行空間が $2^3 = 8$ 元、核 $\ker \mathbf{H}$ が $2^{7-3} = 2^4 = 16$ 元である。実際に数え上げても、行空間 8 語・核 16 語が得られる。

B.4 線形符号と検査行列

ここまでの言葉を符号の言葉に翻訳する。**二元線形符号**とは $\mathbb{F}_2^n$ の部分空間のことであり、その元を**符号語**と呼ぶ。次元が k の符号を $[n, k]$ 符号という。零でない符号語の最小重み d を添えて $[n, k, d]$ と書くこともある (重み = 成分のうち 1 である個数)。

符号 C は、二通りの行列で表せる。

定義 B.5 (生成行列・検査行列) $[n, k]$ 符号 C について、行が C の基底をなす $k \times n$ 行列 G を**生成行列**という。このとき $C = \{ aG : a \in \mathbb{F}_2^k \}$ である。また、 $C = \ker H = \{ x : Hx^\top = 0 \}$ を満たす $(n - k) \times n$ 行列 H を**検査行列**という。検査行列の各行は、符号語が満たすべきパリティ検査一本に対応する。

検査行列の独立な行は $n - k$ 本だから、 $\text{rank } H = n - k$ 、言い換えれば

$$k = n - \text{rank } H$$

である。符号の次元は、検査行列の階数から直ちに読める。

双対符号は、符号と直交するベクトル全体である。

$$C^\perp = \{y \in \mathbb{F}_2^n : y \cdot x = 0 \text{ (すべての } x \in C)\},$$

ここで $y \cdot x = \sum_i y_i x_i \pmod{2}$ は標準内積である。 $C^\perp$ は検査行列 H の行が張る空間、すなわち H の行空間に一致する (H の各行が定義により全符号語と直交するため)。したがって $C^\perp$ の生成行列は H 、検査行列は G であり、

$$\dim C^\perp = \text{rank } H = n - k, \quad \dim C + \dim C^\perp = n.$$

走行例の Steane 検査行列 $\mathbf{H}$ (3×7 , $\text{rank } \mathbf{H} = 3$) では、 $C = \ker \mathbf{H}$ が $[7, 4]$ 符号 (古典 Hamming 符号、 $k = 7 - 3 = 4$ 、16 語)、その双対 $C^\perp$ が $[7, 3]$ 符号 (simplex 符号、 $\mathbf{H}$ の行空間、8 語) である。第 7 章で Steane の 6 本のスタビライザ (X 型 3 + Z 型 3) を、この同じ検査行列 $\mathbf{H}$ の各行を X/Z に置いて構成したのは、 C と $C^\perp$ のこの関係を量子符号に持ち込む操作だった。その持ち込みが整合する条件が、次節の双対包含である。

B.5 双対包含と自己双対

検査行列 H から CSS 量子符号を作るとき (第 7 章)、 X 型生成子と Z 型生成子が互いに可換であることが要る。この可換性を古典符号の言葉で言い換えたものが、双対包含である。

命題 B.2 (双対包含の判定) 検査行列 H の定める符号 $C = \ker H$ について、双対符号が符号に含まれること $C^\perp \subseteq C$ は、

$$HH^\top = 0 \pmod{2}$$

と同値である。

証明. $C^\perp$ は H の行空間だから、 $C^\perp \subseteq C = \ker H$ は「 H の各行 h が $\ker H$ に入る」、すなわち $Hh^\top = 0$ がすべての行で成り立つことと同値である。これらを行ごとに束ねた等式が $HH^\top = 0 \pmod{2}$ にほかならない。 $\square$

判定は、たった一つの行列積で済む。

```

1 import numpy as np
2
3 H = np.array([[0, 0, 0, 1, 1, 1, 1],
4               [0, 1, 1, 0, 0, 1, 1],
5               [1, 0, 1, 0, 1, 0, 1]])
6 # Dual containment  C^perp subset C  <=>  H H^T = 0 (mod 2)
7 M = (H @ H.T) % 2
8 print("H H^T mod 2 =")
9 print(M)
10 print("all zero (dual-containing):", not M.any())

```

```

H H^T mod 2 =
[[0 0 0]
 [0 0 0]
 [0 0 0]]
all zero (dual-containing): True

```

Steane の検査行列はこの判定を満たす ($\mathbf{H}\mathbf{H}^T = 0$)。したがって simplex 符号 $[7, 3]$ は Hamming 符号 $[7, 4]$ に含まれ、第 6 章の「重なり偶数則」が保証していた X 型・ Z 型生成子の可換性が、ここでは $C^\perp \subseteq C$ という一行の条件として現れる。

注意 B.1 (「自己双対」という語の二義) 厳密な意味の自己双対符号は $C = C^\perp$ を満たす符号で、このとき $\dim C = n/2$ が必要だから、長さ n は偶数でなければならない。走行例の Hamming $[7, 4]$ は $C^\perp \subsetneq C$ (次元 $3 \subsetneq 4$) であり、双対包含は満たすが厳密な自己双対ではない (そもそも長さ 7 は奇数で、長さ 7 の自己双対符号は存在しない)。第 7 章以降で Steane を「自己双対 CSS」と呼ぶのは、 X 用と Z 用に同じ符号 $C_1 = C_2 = \text{Hamming}$ を使うことで $X \leftrightarrow Z$ 対称な量子符号が得られることを指しており、古典 Hamming それ自身が $C = C^\perp$ である、という意味ではない。本付録では、CSS 構成に必要な条件は一貫して双対包含 $C^\perp \subseteq C$ である。

付録 C

演習解答

本付録は、第 II 部各章の章末演習の解答をまとめる。すべて理想実行（誤りなし）に絞り、各解答は方針の説明・実装コード（NumPy）・実行出力・結果の読み解きの順で示す。コードは本文と独立に、その演習だけで閉じて動くように書いてある。掲載コードのコメントと出力は英語、解説は日本語とした。数値はすべて独立な計算で検証してある。

C.1 第 6 章

演習 6-1（パウリの GF(2) 表現と可換性のシンプレクティック判定）

パウリ演算子を $(x | z)$ の GF(2) ベクトルに直し、シンプレクティック内積 $x \cdot z' + z \cdot x' \pmod{2}$ が 0 で可換・1 で反可換になることを、三量子ビットのいくつかの組で確かめる。あわせて Shor の八生成子が、任意の二本で内積 0、すなわち互いにすべて可換であることを見る。

```

1 import numpy as np
2 # Pauli string -> GF(2) symplectic vector (x/z): x_i=1 if X/Y, z_i=1
   if Z/Y
3 def symp(s):
4     n=len(s); x=np.zeros(n,int); z=np.zeros(n,int)
5     for i,c in enumerate(s):
6         if c in "XY": x[i]=1
7         if c in "ZY": z[i]=1
8     return np.concatenate([x,z])
9 # symplectic inner product mod 2: 0 = commute, 1 = anticommute
10 def sip(a,b):
11     n=len(a)//2; return int((a[:n]@b[n:]+a[n:]@b[:n])%2)
12 print(f"{'pair':12s}{'sip':>4s} relation")
13 for a,b in [("ZZI","IZZ"),("XII","ZII"),("XXI","ZZI"),("XYZ","ZYX")]:
14     s=sip(symp(a),symp(b))
15     print(f"{{a}},{{b:<5s}}{{s:>5d}}  {'commute' if s==0 else 'anticommute'}")
16 # Shor's eight generators: every pair must have symplectic product 0

```

```

17 shor=[symp(s) for s in ["ZZIIIIII","IZZIIIIII","IIIZZIIII","IIIIZZIII
    ",
18             "IIIIIIZZI","IIIIIIIZZ","XXXXXXIII","IIIXXXXXX
                "]]
19 print("Shor 8 generators all commute:",
20       all(sip(shor[i],shor[j])==0 for i in range(8) for j in range
            (8)))

```

出力:

pair	sip	relation
ZZI,IZZ	0	commute
XII,ZII	1	anticommute
XXI,ZZI	0	commute
XYZ,ZYX	0	commute

Shor 8 generators all commute: True

内積は「一方の X 成分ともう一方の Z 成分が重なる桁数」を mod 2 で数えたもので、 X - Z が交わる桁ごとに反交換の符号が一つ立つ、という勘定そのものだ。XXI と ZZI は二桁で交わるが偶数ゆえ可換、XII と ZII は一桁だけ交わるので反可換になる。Shor の八生成子は、どの二本も偶数桁でしか交わらないよう組まれており、内積はすべて 0——「測定どうしが互いを乱さない」ことの GF(2) 版が、この一つの数え上げに収まっている。

演習 6-2 (符号空間の射影子と次元 2^{n-s})

独立な生成子から符号空間の射影子 $\Pi = \prod_i (1 + g_i)/2$ を組み、その階数 (= 符号空間の次元) を数える。射影子の階数はトレースに等しいので、生成子を一本ずつ掛けるたびのトレースを追えば、次元が半分ずつ落ちて最後に 2^{n-s} に達する様子が見える。三量子ビット片 $\langle Z_1 Z_2, Z_2 Z_3 \rangle$ と Shor の八生成子で確かめる。

```

1 import numpy as np
2 I2=np.eye(2); X=np.array([[0,1],[1,0]]); Z=np.array([[1,0],[0,-1]])
3 def op(s):                                     # Pauli string -> dense
    matrix
4     M=np.array([[1.0]])
5     for c in s: M=np.kron(M,{"I":I2,"X":X,"Z":Z}[c])
6     return M
7 def proj_ranks(gens):                         # trace of running product of
    (I+g)/2 = rank
8     n=len(gens[0]); dim=2**n; P=np.eye(dim); out=[dim]
9     for g in gens:
10        P=P@(np.eye(dim)+op(g))/2; out.append(int(round(np.trace(P))))
11    return out
12 print("3-qubit <Z1Z2,Z2Z3> ranks:", proj_ranks(["ZZI","IZZ"]), "-> 2 =
    2^(3-2)")

```

```

13  shor=["ZZIIIIIII","IZZIIIIIII","IIIZZIIIII","IIIIZZIIII",
14       "IIIIIIZZI","IIIIIIIZZ","XXXXXXXXIII","IIIXXXXXX"]
15  r=proj_ranks(shor)
16  print("Shor 8 generators ranks:", r)
17  print("code-space dimension =", r[-1], "= 2^(9-8) = 2 (not 1)")

```

出力:

```

3-qubit <Z1Z2,Z2Z3> ranks: [8, 4, 2] -> 2 = 2^(3-2)
Shor 8 generators ranks: [512, 256, 128, 64, 32, 16, 8, 4, 2]
code-space dimension = 2 = 2^(9-8) = 2 (not 1)

```

トレースの列は三量子ビットで $8 \rightarrow 4 \rightarrow 2$ 、Shor で 512 から一本ごとに半分に落ちて最後に 2 になる。独立な生成子を一本加えるたびに $+1$ 固有空間が半分に切られ、 s 本で $2^n/2^s = 2^{n-s}$ が残る。Shor は $n = 9 \cdot s = 8$ なので符号空間は $2^{9-8} = 2$ 次元、論理量子ビット一個ぶんである。ここは $2^{9-8} = 1$ ではなく 2 であることに注意——指数の差 $n-s = 1$ がそのまま論理量子ビット数 $k = 1$ 、その状態空間の次元 $2^k = 2$ を与えている。

演習 6-3 (論理演算子 $N(S) \setminus S$ の判定)

パウリ演算子が論理演算子か—— S のすべての生成子と可換で、かつ S に属さないか——を判定する。可換性は演習 6-1 のシンプレクティック内積で、 S への所属は $(x|z)$ ベクトルが生成子ベクトルの $\text{GF}(2)$ 張る空間に入るかで見える。三量子ビット片 $\langle Z_1Z_2, Z_2Z_3 \rangle$ について、 $X_1X_2X_3 \cdot Z_1 \cdot Z_1Z_2 \cdot X_1$ を分類する。

```

1  import numpy as np
2  def symp(s):
3      n=len(s); x=np.zeros(n,int); z=np.zeros(n,int)
4      for i,c in enumerate(s):
5          if c in "XY": x[i]=1
6          if c in "ZY": z[i]=1
7      return np.concatenate([x,z])
8  def sip(a,b):
9      n=len(a)//2; return int((a[:n]@b[n:]+a[n:]@b[:n])%2)
10 def gf2_rank(M):
11     M=(np.array(M,int)%2).copy(); r=0
12     for c in range(M.shape[1]):
13         p=next((i for i in range(r,M.shape[0]) if M[i,c]),None)
14         if p is None: continue
15         M[[r,p]]=M[[p,r]]
16         for i in range(M.shape[0]):
17             if i!=r and M[i,c]: M[i]=(M[i]+M[r])%2
18         r+=1
19     return r
20 def in_S(G,v): return gf2_rank(G)==gf2_rank(np.vstack([G,v]))
21 G=[symp("ZZI"),symp("IZZ")] # S = <Z1Z2, Z2Z3> on three

```

```

qubits
22 print(f"{'op':5s}{'commutes':>9s}{'in S':>8s}    class")
23 for s in ["XXX","ZII","ZZI","XII"]:
24     v=symp(s); c=all(sip(v,g)==0 for g in G); i=in_S(G,v)
25     k="logical operator" if (c and not i) else ("in S (trivial)" if i
        else "not in N(S)")
26     print(f"{s:5s}{str(c):>9s}{str(i):>8s}    {k}")

```

出力:

op	commutes	in S	class
XXX	True	False	logical operator
ZII	True	False	logical operator
ZZI	True	True	in S (trivial)
XII	False	False	not in N(S)

$X_1X_2X_3$ と Z_1 はすべての生成子と可換だが S には属さない——符号空間を保ったまま論理状態を動かす、れっきとした論理演算子だ。 Z_1Z_2 も可換だが S の元なので、論理状態には何もしない自明な作用しか持たない。 X_1 は Z_2Z_3 とは可換でも Z_1Z_2 と反可換で、 $N(S)$ にすら入らない——符号空間の外へ状態を蹴り出してしまふ。論理演算子とは「 S と可換」かつ「 S の外」という二条件で切り出されるこの中間層 $N(S) \setminus S$ にほかならない。

C.2 第7章

演習 7-1 (Hamming のシンドロームが誤りの番地を指すこと)

検査行列 $\mathbf{H}$ (3×7) を用意し、七通りの一ビット誤り e_j について $s = \mathbf{H}e_j$ を mod 2 で計算する。各シンドロームが j の二進表示 (= $\mathbf{H}$ の j 列目) に一致し、七つのシンドロームが 000 を除く三ビットパターンを一度ずつ取ることを確かめる。

```

1 import numpy as np
2 H=np.array([[0,0,0,1,1,1,1],[0,1,1,0,0,1,1],[1,0,1,0,1,0,1]]) # cols
   = binary 1..7
3 print(f"{'error':6s}{'syndrome':>9s}{'int':>5s} ==j?")
4 seen=set()
5 for j in range(7):
6     e=np.zeros(7,int); e[j]=1; s=(H@e)%2
7     val=int(s[0]*4+s[1]*2+s[2]); seen.add(tuple(s))
8     print(f"e{j+1:<5d}{''.join(map(str,s)):>9s}{val:>5d} {'yes' if
        val==j+1 else 'NO'}")
9 print("distinct nonzero syndromes:", len(seen), "of 7")

```

出力:

```
error syndrome int ==j?
```

e1	001	1	yes
e2	010	2	yes
e3	011	3	yes
e4	100	4	yes
e5	101	5	yes
e6	110	6	yes
e7	111	7	yes

distinct nonzero syndromes: 7 of 7

七つのシンδροームは $001, 010, \dots, 111$ と、 j の二進表示をちょうど一度ずつ取った。 $\mathbf{H}e_j$ は定義から $\mathbf{H}$ の j 列目で、その列が j の二進表示になるよう並べてあるから、シンδροームを二進数として読めば反転した桁の番地がそのまま出る。これが古典 Hamming の復号——症状の三ビットを数に直して誤り位置を名指す——の中身で、量子側でこのシンδροームを Z 型・ X 型生成子の測定として読むのが第7章の構成だった。

演習 7-2 (双対包含 $\text{Hamming}^\perp \subseteq \text{Hamming}$ の確認)

$\mathbf{H}$ の三行が $\text{GF}(2)$ 上で張る空間——双対符号 $\text{Hamming}^\perp$ ($[7, 3]$ simplex、八語)——を生成し、その各語 v が $\mathbf{H}v = 0 \pmod{2}$ を満たす、すなわち Hamming の語でもあることを見る。同じことが行どうしの直交 $\mathbf{H}\mathbf{H}^T = 0 \pmod{2}$ にまともすることも直接確かめる。

```

1 import numpy as np, itertools
2 H=np.array([[0,0,0,1,1,1,1],[0,1,1,0,0,1,1],[1,0,1,0,1,0,1]])
3 # dual code C^perp = GF(2) row span of H (8 words, simplex [7,3])
4 Cperp={tuple((np.array(c)@H)%2) for c in itertools.product([0,1],
5     repeat=3)}
6 print("|C^perp| =", len(Cperp), "(expected 8 = 2^3)")
7 print("every dual word v has H v = 0 (mod 2):",
8     all(not ((H@np.array(v))%2).any() for v in Cperp))
9 print("H H^T (mod 2) =\n", (H@H.T)%2)
10 print("H H^T == 0 (mod 2):", not ((H@H.T)%2).any())

```

出力:

```

|C^perp| = 8 (expected 8 = 2^3)
every dual word v has H v = 0 (mod 2): True
H H^T (mod 2) =
[[0 0 0]
 [0 0 0]
 [0 0 0]]
H H^T == 0 (mod 2): True

```

双対符号の八語はすべて $\mathbf{H}v = 0$ を満たし、Hamming 符号の語でもある——双対包含

$\text{Hamming}^\perp \subseteq \text{Hamming}$ が成り立っている。同じ事実は $\mathbf{H}\mathbf{H}^\top = 0 \pmod{2}$ という一つの行列等式に凝縮される。この等式こそが、 X 型生成子と Z 型生成子を同じ $\mathbf{H}$ から組んだとき六本が互いに可換になる根拠であり、第 6 章の「重なり偶数則」を古典符号の言葉で言い直したものである。

演習 7-3（検査行列から生成子を組み、可換性を数値で確かめる）

演習 6-1 のシンプレクティック表現で、 $\mathbf{H}$ の三行から X 型 3 本・ Z 型 3 本の生成子を組む。六本の任意の二本で内積が 0（互いに可換）になること、さらに $X_L = X^{\otimes 7} \cdot Z_L = Z^{\otimes 7}$ が六本すべてと可換で、互いには反可換であることを確かめる。

```

1 import numpy as np
2 H=np.array([[0,0,0,1,1,1,1],[0,1,1,0,0,1,1],[1,0,1,0,1,0,1]])
3 def sip(a,b):
4     n=len(a)//2; return int((a[:n]@b[n:]+a[n:]@b[:n])%2)
5 z=np.zeros(7,int)
6 # Steane generators in (x/z): X-type = [row|0], Z-type = [0|row]
7 G=[np.concatenate([r,z]) for r in H]+[np.concatenate([z,r]) for r in H
8     ]
9 M=np.array([[sip(G[i],G[j]) for j in range(6)] for i in range(6)])
10 print("pairwise symplectic products (0 = commute):\n", M)
11 print("all six generators commute:", not M.any())
12 o=np.ones(7,int); XL=np.concatenate([o,z]); ZL=np.concatenate([z,o])
13 print("X_L=X^7 commutes with all six:", all(sip(XL,g)==0 for g in G))
14 print("Z_L=Z^7 commutes with all six:", all(sip(ZL,g)==0 for g in G))
15 print("X_L, Z_L anticommute:", sip(XL,ZL)==1)

```

出力:

```
pairwise symplectic products (0 = commute):
```

```

[[0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]]

```

```
all six generators commute: True
```

```
X_L=X^7 commutes with all six: True
```

```
Z_L=Z^7 commutes with all six: True
```

```
X_L, Z_L anticommute: True
```

六本の生成子のシンプレクティック内積はすべて 0、 6×6 の零行列になった。 X 型どうし・ Z 型どうしは成分が片側にしかないので自動的に可換、 X 型と Z 型の交差項は $\mathbf{H}$ の行どうしの内積 $\mathbf{H}\mathbf{H}^\top$ （演習 7-2 で 0）に一致して可換になる。論理演算子 $X_L = X^{\otimes 7} \cdot Z_L = Z^{\otimes 7}$

は六本すべてと可換（符号空間を保つ）でありながら互いには反可換——量子ビットぶんのパウリ代数を符号空間の上に正しく実現する代表になっている。

C.3 第8章

演習 8-1（論理 0 の構成と全スタビライザ $+1$ の確認）

論理 0 を組み、八項であること、そして六本の生成子すべてに $+1$ で固定されることを数値で確かめる。経路は二つある。第一は、演習 7-2 で生成した双対符号 $C^\perp$ の八語を、そのまま

$$|0_L\rangle = \frac{1}{\sqrt{8}} \sum_{v \in C^\perp} |v\rangle$$

として足し上げる素直なやり方だ。続いて、各検査行を X （あるいは Z ）に乗せて X 型・ Z 型の生成子を 7 量子ビットの演算子として組み、期待値 $\langle 0_L | g | 0_L \rangle$ を六本ぶん計算する。

```

1 import numpy as np
2 import itertools
3
4 # Hamming check matrix H (columns = binary 1..7, lexicographic)
5 H = np.array([[0, 0, 0, 1, 1, 1, 1],
6               [0, 1, 1, 0, 0, 1, 1],
7               [1, 0, 1, 0, 1, 0, 1]])
8
9 # Dual code C^perp: GF(2) row span of H = 8 words (Exercise 7-2)
10 Cperp = sorted({tuple((np.array(a) @ H) % 2)
11                 for a in itertools.product([0, 1], repeat=3)})
12
13 # Pauli operators and their embedding into 7 qubits
14 I2 = np.eye(2); X = np.array([[0, 1], [1, 0]]); Z = np.array([[1, 0],
15                        [0, -1]])
16 def kron(ms):
17     out = np.array([[1.0]])
18     for m in ms:
19         out = np.kron(out, m)
20     return out
21 def ket(bits):                                     # |bits> (length-7
22     0/1 list)
23     v = np.zeros(2 ** 7)
24     v[int("".join(map(str, bits)), 2)] = 1.0
25     return v
26 def Xrow(row): return kron([X if b else I2 for b in row]) # check
27 row -> X-type gen
28 def Zrow(row): return kron([Z if b else I2 for b in row]) # check
29 row -> Z-type gen
30
31 # Logical zero = equal superposition of the 8 dual-code words

```

```

28 zero_L = sum(ket(v) for v in Cperp) / np.sqrt(8)
29
30 # (1) nonzero amplitudes: addresses and Hamming weights
31 nz = np.where(np.abs(zero_L) > 1e-9)[0]
32 print("number of nonzero amplitudes:", len(nz))
33 for i in nz:
34     w = format(i, "07b")
35     print(f"  |{w}>  weight {w.count('1')}")
36
37 # (2) fixed at +1 by each of the 6 generators? <0_L|g|0_L>
38 gens = [("X1", Xrow(H[0])), ("X2", Xrow(H[1])), ("X3", Xrow(H[2])),
39         ("Z1", Zrow(H[0])), ("Z2", Zrow(H[1])), ("Z3", Zrow(H[2]))]
40 print("expectation <0_L|g|0_L>:")
41 for name, g in gens:
42     print(f"  {name}: {np.real(zero_L @ g @ zero_L):+.4f}")
43
44 # (3) (alt.) projector prod (I + g_i^X)/2 on |0000000> = same state
45 P = np.eye(2 ** 7)
46 for row in H:
47     P = P @ (np.eye(2 ** 7) + Xrow(row)) / 2
48 proj = P @ ket([0] * 7)
49 proj = proj / np.linalg.norm(proj)
50 print("matches projector route:", bool(np.allclose(proj, zero_L)))

```

出力:

number of nonzero amplitudes: 8

```

|0000000>  weight 0
|0001111>  weight 4
|0110011>  weight 4
|0111100>  weight 4
|1010101>  weight 4
|1011010>  weight 4
|1100110>  weight 4
|1101001>  weight 4

```

expectation <0_L|g|0_L>:

```

X1: +1.0000
X2: +1.0000
X3: +1.0000
Z1: +1.0000
Z2: +1.0000
Z3: +1.0000

```

matches projector route: True

非零の振幅はちょうど八つ、内訳は重み 0 の $|0000000\rangle$ がひとつと重み 4 の語が七つで、本文 §8.2 で書き下した八項に一致する。六本の生成子の期待値はすべて +1 になり、 $|0_L\rangle$ が符号空間の住人であることが確かめられた。 X 型生成子は八項の顔ぶれを並べ替えるだけ、 Z 型生成子は双対包含 ($C^\perp$ の語どうしの重なり $w \cdot v$ が偶数) によって各項に +1 を返す——この二つがそろって、六本すべてが +1 になる。

コード末尾の別経路、射影子 $\prod_i \frac{1+g_i}{2}$ を $|0000000\rangle$ に施す計算も、規格化すれば同じ状態を返す (出力末尾の True)。本文 §8.2 で「 X 型生成子で均す」と述べた操作は、この射影子そのものだった。項の数がなぜちょうど八か——独立な三つの反転が番地を $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$ と倍々にするから——は一般化節 §8.4 で扱っている。

演習 8-2 (論理 1 の構成と軽い論理測定による直交の確認)

論理 1 を作り、軽い論理測定で論理 0 と直交することを確認する。論理 1 は、前章で代表に選んだ論理 X 、すなわち $X_L = X^{\otimes 7}$ を $|0_L\rangle$ に施して得る—— $X^{\otimes 7}$ は各番地のビットをすべて裏返すので、 $|0_L\rangle$ の八語はその全ビット補集合へ移る。続いて「軽い論理測定」を、全量子ビットを Z 基底で測ったときの 0/1 列のパリティ (1 の個数の偶奇) として実装する——これは論理 Z 、 $Z_L = Z^{\otimes 7}$ の固有値そのものだ。最後に内積 $\langle 0_L | 1_L \rangle$ を直接計算する。

```

1 import numpy as np
2 import itertools
3
4 # Steane code: dual code  $C^\perp$  (8 words) and logical zero (Exercise 8-1)
5 H = np.array([[0, 0, 0, 1, 1, 1, 1],
6               [0, 1, 1, 0, 0, 1, 1],
7               [1, 0, 1, 0, 1, 0, 1]])
8 Cperp = sorted({tuple((np.array(a) @ H) % 2)
9                  for a in itertools.product([0, 1], repeat=3)})
10 I2 = np.eye(2); X = np.array([[0, 1], [1, 0]]); Z = np.array([[1, 0],
11                       [0, -1]])
12 def kron(ms):
13     out = np.array([[1.0]])
14     for m in ms:
15         out = np.kron(out, m)
16     return out
17 def ket(bits):
18     v = np.zeros(2 ** 7)
19     v[int("".join(map(str, bits)), 2)] = 1.0
20     return v
21 def support(vec):
22     # addresses with nonzero amplitude
23     return [format(i, "07b") for i in np.where(np.abs(vec) > 1e-9)[0]]
24
25 zero_L = sum(ket(v) for v in Cperp) / np.sqrt(8)

```

```

24 XL = kron([X] * 7)                                # logical X =  $X^{\sim}\{x$ 
    )7}
25 ZL = kron([Z] * 7)                                # logical Z =  $Z^{\sim}\{x$ 
    )7}
26 one_L = XL @ zero_L                                # logical 1 =  $X^{\sim}\{x$ 
    )7} |0_L>
27
28 # (1) |1_L> lives on the bitwise complements of the |0_L> words
29 print("|1_L> support (address, weight):")
30 for w in support(one_L):
31     print(f"    |{w}>    weight {w.count('1')}")
32
33 # (2) light logical measurement: parity of Z-basis outcome =
    eigenvalue of Z_L
34 print("parity set of support words:")
35 print("    |0_L>:", sorted({w.count('1') % 2 for w in support(zero_L)}),
    "-> even -> logical 0")
36 print("    |1_L>:", sorted({w.count('1') % 2 for w in support(one_L)}),
    "-> odd  -> logical 1")
37 print(f"    <0_L|Z_L|0_L> = {np.real(zero_L @ ZL @ zero_L):+.4f}    "
    f"<1_L|Z_L|1_L> = {np.real(one_L @ ZL @ one_L):+.4f}")
38
39
40 # (3) orthogonality of the two logical states
41 print(f"<0_L|1_L> = {np.real(np.vdot(zero_L, one_L)):+.4f}")

```

出力:

```

|1_L> support (address, weight):
|0010110> weight 3
|0011001> weight 3
|0100101> weight 3
|0101010> weight 3
|1000011> weight 3
|1001100> weight 3
|1110000> weight 3
|1111111> weight 7
parity set of support words:
|0_L>: [0] -> even -> logical 0
|1_L>: [1] -> odd  -> logical 1
<0_L|Z_L|0_L> = +1.0000    <1_L|Z_L|1_L> = -1.0000
<0_L|1_L> = +0.0000

```

$|1_L\rangle$ の八項はすべて奇数重み——重み 7 の $|1111111\rangle$ がひとつと重み 3 の語が七つ——で、 $|0_L\rangle$ の偶数重みの八項（重み 0 と重み 4）とは番地をひとつも共有しない。したがって内積

は0、両者は直交する（出力末尾の $\langle 0_L | 1_L \rangle$ ）。軽い測定では $|0_L\rangle$ の支持がすべて偶パリティ、 $|1_L\rangle$ がすべて奇パリティとなり、これはちょうど Z_L の固有値 $+1$ （論理0）と -1 （論理1）に対応する。シンドロームを取って訂正する手続きを経ずとも、全量子ビットを Z で測ってパリティを読むだけで二つの論理状態が見分けられる。ただしこれは誤りのない理想実行に限った話で、途中で物理量子ビットに誤りが乗れば測定列は八語から外れ、この軽い読み出しは破れる——その場合の正しい論理測定は次章のフル版で扱う。

演習 8-3（論理 + の構成と論理パウリ代数の確認）

論理 $+$ を置き、論理ビットがひとつ分のパウリ代数を回すことを確かめる。 $|+_L\rangle = (|0_L\rangle + |1_L\rangle)/\sqrt{2}$ を構成し、論理 X 、すなわち $X^{\otimes 7}$ の $+1$ 固有状態 $X^{\otimes 7} |+_L\rangle = |+_L\rangle$ になっていることを見る。あわせて $X_L = X^{\otimes 7}$ と $Z_L = Z^{\otimes 7}$ が反可換 ($X_L Z_L = -Z_L X_L$) で、符号空間の上で $|0_L\rangle$ と $|1_L\rangle$ をちょうど一量子ビットのパウリのように動かすことを確認する。最後に二つの〔発展〕—— $|+_L\rangle$ を計算基底で書き下すと元の $[7, 4]$ Hamming 符号の十六語に均等に散ること、および論理 X が S を法として代表に自由であること——を数値で見る。

```

1  import numpy as np
2  import itertools
3
4  # Steane code setup (as in Exercises 8-1, 8-2)
5  H = np.array([[0, 0, 0, 1, 1, 1, 1],
6                [0, 1, 1, 0, 0, 1, 1],
7                [1, 0, 1, 0, 1, 0, 1]])
8  Cperp = sorted({tuple((np.array(a) @ H) % 2)
9                  for a in itertools.product([0, 1], repeat=3)})
10                 # dual code [7,3], 8 words
11 C = sorted({"".join(map(str, b)) for b in itertools.product([0, 1],
12                     repeat=7)
13             if not (H @ np.array(b) % 2).any()})
14                 # Hamming code [7,4], 16 words
15
16 I2 = np.eye(2); X = np.array([[0, 1], [1, 0]]); Z = np.array([[1, 0],
17                       [0, -1]])
18
19 def kron(ms):
20     out = np.array([[1.0]])
21     for m in ms:
22         out = np.kron(out, m)
23     return out
24
25 def ket(bits):
26     v = np.zeros(2 ** 7)
27     v[int("".join(map(str, bits)), 2)] = 1.0
28     return v
29
30 def support(vec):
31     return [format(i, "07b") for i in np.where(np.abs(vec) > 1e-9)[0]]

```

```

24
25 zero_L = sum(ket(v) for v in Cperp) / np.sqrt(8)
26 XL = kron([X] * 7) # logical X = X^{(x)}
27 ZL = kron([Z] * 7) # logical Z = Z^{(x)}
28 one_L = XL @ zero_L
29 plus_L = (zero_L + one_L) / np.sqrt(2) # logical + = (|0_L> + |1_L>)/sqrt2
30
31 # (1) |+_L> is the +1 eigenstate of logical X
32 print("X^{(x)} |+_L> == |+_L> :", np.allclose(XL @ plus_L,
33         plus_L))
34 # (2) logical X and Z anticommute and act as a single-qubit Pauli on
35 # |0_L>, |1_L>
36 print("X_L Z_L == -Z_L X_L :", np.allclose(XL @ ZL, -ZL @
37         XL))
38 print("X_L: |0_L><->|1_L> :",
39         np.allclose(XL @ zero_L, one_L) and np.allclose(XL @ one_L,
40         zero_L))
41 print("Z_L: |0_L>->+|0_L>, |1_L>->-|1_L>:",
42         np.allclose(ZL @ zero_L, zero_L) and np.allclose(ZL @ one_L, -
43         one_L))
44
45 # (3) [extension] |+_L> spreads over the 16 words of the Hamming code
46 # C=[7,4]
47 supp = support(plus_L)
48 amps = np.abs(plus_L[np.abs(plus_L) > 1e-9])
49 print("|+_L> support size :", len(supp), "(Hamming C:",
50         len(C), "words)")
51 print("|+_L> support == Hamming code C :", supp == C)
52 print("all |+_L> amplitudes == 1/4 :", np.allclose(amps, 1 / 4))
53
54 # (4) [extension] logical X is free modulo S: X^{(x)} * g == X^{(x)} on
55 # the code space
56 gens = [kron([X if b else I2 for b in r]) for r in H] \
57         + [kron([Z if b else I2 for b in r]) for r in H]
58 print("X^{(x)} * (any of 6 gens) == X_L :",
59         all(np.allclose((XL @ g) @ zero_L, XL @ zero_L) for g in gens))
60 v = set(int(i) for i in np.where(H[0] == 1)[0])
61 alt = [q for q in range(7) if q not in v] # complement of an
62 # X-type support, weight 3
63 Xw3 = kron([X if q in alt else I2 for q in range(7)])
64 print(f"weight-{len(alt)} rep (X on {alt}) acts as X_L:",
65         np.allclose(Xw3 @ zero_L, one_L) and np.allclose(Xw3 @ one_L,
66         zero_L))

```

出力:

```
X^(x)7 |+_L> == |+_L>          : True
X_L Z_L == -Z_L X_L            : True
X_L: |0_L><->|1_L>              : True
Z_L: |0_L>->+|0_L>, |1_L>->-|1_L>: True
|+_L> support size              : 16 (Hamming C: 16 words)
|+_L> support == Hamming code C : True
all |+_L> amplitudes == 1/4    : True
X^(x)7 * (any of 6 gens) == X_L : True
weight-3 rep (X on [0, 1, 2]) acts as X_L: True
```

$|+_L\rangle$ が論理 X の $+1$ 固有状態になるのは定義からすぐで、 $X^{\otimes 7}$ が $|0_L\rangle$ と $|1_L\rangle$ を入れ替えるため、その和は不変に保たれる。 X_L と Z_L は反可換で、符号空間に限れば X_L が $|0_L\rangle \leftrightarrow |1_L\rangle$ を交換し、 Z_L が $|0_L\rangle$ に $+1$ 、 $|1_L\rangle$ に -1 を付ける——物理は七量子ビットだが、論理層には一量子ビットのパウリ X 、 Z がそのまま現れる。これが「七つの物理ビットに埋め込んだ一つの論理ビット」という言い方の具体的な中身である。

〔発展〕の一つ目は $|0_L\rangle$ と $|+_L\rangle$ の双対性だ。 $|0_L\rangle$ は双対符号 $[7, 3]$ の八語に散ったが（演習 8-1）、 $|+_L\rangle$ は元の Hamming 符号 $[7, 4]$ の十六語に、すべて振幅 $1/4$ で均等に散る（出力の support size 16、全振幅 $1/4$ ）。 Z 基底で八語に固まって見える状態が X 基底では十六語に広がり、その逆も成り立つ——CSS 構成が X と Z を対称に扱っていることの現れである。

〔発展〕の二つ目は論理演算子の代表の自由さだ。 $X^{\otimes 7}$ に安定化群 S の生成子をどれか一本掛けても、符号空間の上での作用は変わらない（出力の該当行が True）。実際、 X 型生成子の台の補集合にあたる重み 3 の演算子——ここでは量子ビット 0, 1, 2 に作用する X ——も、まったく同じ論理 X として働く。論理 X は「 $X^{\otimes 7}$ 」という特定の一個ではなく、 S を法とした同値類であって、重い代表でも軽い代表でも論理的には同一だ——フォールトトレラント実装で軽い代表を選びたくなる動機が、ここに芽生えている。

C.4 第9章

演習 9-1 (transversal CNOT の誤り非伝播)

観察 2 で図にした「transversal CNOT は誤りを兄弟に渡さない」を、数値で裏取りする。二つの符号ブロック A （量子ビット 0–6）と B （量子ビット 7–13）の計 14 量子ビットを、パウリ枠 $(x|z)$ —— 各桁に X/Y があれば $x = 1$ 、 Z/Y があれば $z = 1$ とする二値ベクトル——で表す。 A の量子ビット i と B の量子ビット i を結ぶ transversal CNOT を通し、制御ブロックの一量子ビットに置いた X 誤り（および標的側に置いた Z 誤り）の台がどう動くかを見る。

```
1 import numpy as np
2
```

```

3 # Two Steane blocks A (qubits 0..6) and B (qubits 7..13), 14 qubits
  total.
4 # Track a Pauli error as a binary frame (x/z) over 14 qubits:
5 #   x[q]=1 if X or Y on qubit q,   z[q]=1 if Z or Y on qubit q.
6 N = 14
7 def cnot(x, z, c, t):                                # CNOT: X_c -> X_c X_t,   Z_t ->
  Z_c Z_t
8     x = x.copy(); z = z.copy()
9     x[t] ^= x[c]; z[c] ^= z[t]
10    return x, z
11 def transversal_cnot(x, z):                          # qubit i of A controls qubit i
  of B (i = 0..6)
12     for i in range(7):
13         x, z = cnot(x, z, i, 7 + i)
14     return x, z
15 def block_weights(x, z):
16     s = [(x[q] | z[q]) for q in range(N)]
17     return int(sum(s[0:7])), int(sum(s[7:14]))
18 def support(x, z):
19     out = []
20     for q in range(N):
21         if x[q] and z[q]: out.append(f"Y{q}")
22         elif x[q]:        out.append(f"X{q}")
23         elif z[q]:        out.append(f"Z{q}")
24     return out
25
26 # (1) single X error on qubit 2 of control block A, then transversal
  CNOT
27 x = np.zeros(N, int); z = np.zeros(N, int); x[2] = 1
28 x, z = transversal_cnot(x, z)
29 print("X on A.qubit2 -> support:", support(x, z), " block weights (A,
  B):", block_weights(x, z))
30
31 # (2) single Z error on qubit 4 of target block B, then transversal
  CNOT
32 x = np.zeros(N, int); z = np.zeros(N, int); z[7 + 4] = 1
33 x, z = transversal_cnot(x, z)
34 print("Z on B.qubit4 -> support:", support(x, z), " block weights (A,
  B):", block_weights(x, z))

```

出力:

```

X on A.qubit2 -> support: ['X2', 'X9']  block weights (A,B): (1, 1)
Z on B.qubit4 -> support: ['Z4', 'Z11'] block weights (A,B): (1, 1)

```

制御ブロック A の量子ビット 2 に置いた X 誤りは、CNOT を通すと標的ブロック B の対応する量子ビット（全体での番地 9）へ複製され、台は $\{X_2, X_9\}$ になる。A・B いずれの

ブロックも weight は 1 のまま —— 誤りは相方の一個へ移るだけで、同一ブロックの兄弟には乗らない。標的側 B の量子ビット 4 に置いた Z 誤りも対称で、制御側 A の対応する量子ビットへ移って台 $\{Z_4, Z_{11}\}$ となり、やはり各ブロック weight 1。CNOT は X を制御から標的へ、 Z を標的から制御へ伝えるが、transversal 配線では「対応する一個」どうししか繋がっていないため、一個の故障は各ブロック高々一個の誤りに留まる —— これが距離 3 の保証を壊さない仕組みである。

演習 9-2 (非 transversal CNOT による weight 2 への増殖と訂正の破綻)

今度は逆を、自分の手で壊して確かめる。同一ブロックの中の二量子ビット —— 量子ビット 2 を制御、量子ビット 4 を標的 —— に物理 CNOT を当てる。制御側に置いた一個の X 誤りがブロック内で weight 2 に育つこと、さらにその weight 2 誤りが単一誤り訂正表では正しく直せず、論理演算子と区別がつかなくなることを見る。 X 誤りは Z 型検査 H で捕まるので、シンドロームは H を掛けて読む。

```

1 import numpy as np
2 import itertools
3
4 # Steane Hamming check matrix H (columns = binary 1..7, lexicographic;
   same as ch7)
5 H = np.array([[0,0,0,1,1,1,1],
6               [0,1,1,0,0,1,1],
7               [1,0,1,0,1,0,1]])
8
9 # A NON-transversal CNOT inside one block: control = qubit 2, target =
   qubit 4
10 x = np.zeros(7, int); x[2] = 1
11 x[4] ^= x[2]                                     # in-block CNOT(2 ->
   4) on the X error
12 print("after in-block CNOT(2->4): support =",
13       [f"X{q}" for q in np.where(x == 1)[0]], " weight", int(x.sum()))
14
15 # Syndrome of this weight-2 error (X errors are seen by the Z-type
   checks = H)
16 s = (H @ x) % 2
17 cols = [tuple(H[:, j]) for j in range(7)]
18 print("syndrome", tuple(int(v) for v in s), "= column of qubit",
19       cols.index(tuple(s)) + 1, "(table: flip it)")
20
21 # Apply the correction the single-error table dictates, inspect the
   residual
22 q = s[0]*4 + s[1]*2 + s[2]*1                       # syndrome -> qubit
   number (1..7)
23 corr = np.zeros(7, int); corr[q - 1] = 1
24 res = (x + corr) % 2

```

```

25 print("after mis-correction: residual =",
26       [f"X{q}" for q in np.where(res == 1)[0]], " weight", int(res.sum()
27       ()))
28 print("residual syndrome", tuple(int(v) for v in (H @ res) % 2), "(
29       zero -> undetectable)")
30
31 def in_stab(t):
32     # in Cperp = GF(2)
33     row space of H ?
34     return any(tuple((np.array(a) @ H) % 2) == tuple(t)
35                  for a in itertools.product([0,1], repeat=3))
36 print("residual: stabilizer?", in_stab(res), " odd weight?", bool(res.
37       sum() % 2),
38       "-> logical X" if (not in_stab(res) and res.sum() % 2) else "")

```

出力:

```

after in-block CNOT(2->4): support = ['X2', 'X4'] weight 2
syndrome (1, 1, 0) = column of qubit 6 (table: flip it)
after mis-correction: residual = ['X2', 'X4', 'X5'] weight 3
residual syndrome (0, 0, 0) (zero -> undetectable)
residual: stabilizer? False odd weight? True -> logical X

```

量子ビット 2 の X 誤りは、同一ブロック内 CNOT($2 \rightarrow 4$) で量子ビット 4 へも移り、台は $\{X_2, X_4\} \cdot \text{weight } 2$ になる。この $\text{weight } 2$ 誤りのシンドロームは $(1, 1, 0)$ で、これは単一誤り (量子ビット 6) のシンドロームとちょうど一致してしまう — 訂正表は「量子ビット 6 を直せ」と誤って指示する。指示どおり直すと、残留は $\{X_2, X_4, X_5\} \cdot \text{weight } 3$ 。残留のシンドロームは $(0, 0, 0)$ で検出にかからず、しかもスタビライザではなく奇重み — すなわち論理 X 、論理誤りそのものだ。transversal を捨てた瞬間、最初の一個の故障が二個に育ち、距離 3 の訂正能力が崩れる。観察 2 の対比が、手元で再現された。

演習 9-3 (誤り込みの論理測定と論理ベル対の相関)

最後に、誤り込みの論理測定を実装し、論理ベル対の相関を読む。論理状態を全量子ビット Z 基底で測って得たビット列に Hamming のシンドローム $\mathbf{H}m$ を当て、誤り位置を表で引いて訂正し、全桁の偶奇から論理 Z を読む。各符号語に単一 X 誤りを一個ずつ乗せて、論理 0/1 が正しく読めるかを総当りで確かめる。仕上げて二つの論理量子ビットで論理ベル対を組み、理想実行のもとで $Z_{L,A}Z_{L,B} = X_{L,A}X_{L,B} = +1$ が出ることを見る。状態は全量子ビット行列を作らず、符号語の振幅表とパウリ枠だけで閉じる。

```

1 import numpy as np
2 import itertools
3
4 # Steane code: check matrix, dual code Cperp (8 words), Hamming code
5   C (16 words)

```

```

5 H = np.array([[0,0,0,1,1,1,1],
6               [0,1,1,0,0,1,1],
7               [1,0,1,0,1,0,1]])
8 Cperp = sorted({tuple((np.array(a) @ H) % 2) for a in itertools.
9                  product([0,1], repeat=3)})
10 C = sorted({tuple(b) for b in itertools.product([0,1], repeat=7)
11             if not (H @ np.array(b) % 2).any()})
12
13 # error-laden logical Z measurement = classical decode of the Z-basis
14 # outcome
15 def measure_logical_Z(outcome):
16     s = (H @ outcome) % 2 # syndrome from the
17                             # measured bit string
18     q = s[0]*4 + s[1]*2 + s[2]*1 # syndrome -> qubit
19                                     # number (0 = none)
20     fixed = outcome.copy()
21     if q >= 1: fixed[q - 1] ^= 1 # undo the single
22                                     # error the table points to
23     return parity(fixed) # overall parity =
24                             # logical Z eigenvalue
25
26 zeroL = [np.array(v) for v in Cperp] # logical 0 words (
27                                     # even weight)
28 oneL = [(np.array(v) ^ 1) for v in Cperp] # logical 1 words (
29                                     # bitwise complements)
30 ok0 = all(measure_logical_Z((w ^ np.eye(7, dtype=int)[j]) % 2) == 0
31           for w in zeroL for j in range(7))
32 ok1 = all(measure_logical_Z((w ^ np.eye(7, dtype=int)[j]) % 2) == 1
33           for w in oneL for j in range(7))
34 print("logical 0 read as 0 under any single X error:", ok0)
35 print("logical 1 read as 1 under any single X error:", ok1)
36
37 # logical Bell pair |Phi> = CNOT_L (H_L|0_L> (x) |0_L>) as a 14-bit
38 # word-amplitude table
39 amp = 1 / np.sqrt(16 * 8)
40 state = {}
41 for w in C: # H_L|0_L> = (1/4)
42     sum_{w in C}|w> (block A)
43     for v in Cperp: # |0_L> = (1/sqrt8)
44         sum_{v in C^perp}|v> (B)
45         b = tuple((np.array(v) ^ np.array(w)) % 2) # CNOT_L: |w>|v>
46             -> |w>|v+w>
47         state[(tuple(w), b)] = state.get((tuple(w), b), 0) + amp
48 norm = sum(abs(a)**2 for a in state.values())
49 ZZ = sum(abs(a)**2 * (-1)**(parity(k[0]) + parity(k[1])) for k, a in
50          state.items())

```

```

37 ones = np.ones(7, int); XX = 0
38 for (a, b), c in state.items():
39     af = tuple((np.array(a) ^ ones) % 2); bf = tuple((np.array(b) ^
40         ones) % 2)
41     XX += np.conj(state.get((af, bf), 0)) * c
42 print(f"<Z_LA Z_LB> = {ZZ:+.4f}    <X_LA X_LB> = {XX.real:+.4f}    (norm
    {norm:.4f})")

```

出力:

```

logical 0 read as 0 under any single X error: True
logical 1 read as 1 under any single X error: True
<Z_LA Z_LB> = +1.0000    <X_LA X_LB> = +1.0000    (norm 1.0000)

```

単一 X 誤りを乗せた論理 0 はすべて論理 0、論理 1 はすべて論理 1 と読める（出力の True 二行）—— 測定列にシンドロームを当てて訂正してからパリティを取る、という「量子測定の後ろに古典デコーダを一つ置く」操作が、誤り込みでも正しい論理値を返している。論理ベル対では $\langle Z_{L,A} Z_{L,B} \rangle$ と $\langle X_{L,A} X_{L,B} \rangle$ がともに +1（規格化 1.0000）となり、ベル相関が論理層にそのまま立ち上がる。誤り込みの論理読み出しが古典デコードそのものであることを、測定列への後処理として体感できる —— シンドロームを取り、誤りを直し、論理パリティを読む、その三手が論理測定の中身だった。

C.5 第 10 章

演習 10-1（三符号の比較表と Steane を選ぶ理由）

距離 3 の三符号を、距離以外の量で並べ直す。Shor $[[9, 1, 3]]$ ・五量子ビット $[[5, 1, 3]]$ ・Steane $[[7, 1, 3]]$ の生成子を与え、物理量子ビット数・CSS か否か・生成子の weight・transversal H が論理ゲートになるか ($H^{\otimes n}$ が生成子群を保つか) を NumPy で導出して表に埋める。紙の上で論じる演習だが、表の各列は計算で裏取りできる。

```

1 import numpy as np
2 # Steane generators are built from the Hamming check matrix (same H as
   ch7-ch9);
3 # Shor and five-qubit are given by their standard generator strings.
4 H = np.array([[0,0,0,1,1,1,1],[0,1,1,0,0,1,1],[1,0,1,0,1,0,1]])
5 def symp(s):
6     n=len(s); x=np.zeros(n,int); z=np.zeros(n,int)
7     for i,c in enumerate(s):
8         if c in "XY": x[i]=1
9         if c in "ZY": z[i]=1
10    return np.concatenate([x,z])
11 def steane():
12    G=[np.concatenate([r,np.zeros(7,int)]) for r in H]
13    G+=[np.concatenate([np.zeros(7,int),r]) for r in H]

```

```

14     return G
15 CODES={
16     "Shor  [[9,1,3]]":[symp(s) for s in
17         ["ZZIIIIIII","IZZIIIIIII","IIIZZIIIII","IIIIZZIIII","IIIIIIZZI","
18             IIIIIIZZ",
19             "XXXXXXIII","IIXXXXXX"]],
20     "Five  [[5,1,3]]":[symp(s) for s in ["XZZXI","IXZZX","XIXZZ","ZXIXZ"
21         ]],
22     "Steane[[7,1,3]]":steane()}
23 def weight(v): n=len(v)//2; return int(((v[:n]|v[n:])!=0).sum())
24 def is_pure(v): n=len(v)//2; return (not v[:n].any()) or (not v[n:].
25     any())
26 def rank(M):
27     M=(np.array(M,int)%2).copy(); r=0
28     for c in range(M.shape[1]):
29         p=next((i for i in range(r,M.shape[0]) if M[i,c]),None)
30         if p is None: continue
31         M[[r,p]]=M[[p,r]]
32         for i in range(M.shape[0]):
33             if i!=r and M[i,c]: M[i]=(M[i]+M[r])%2
34         r+=1
35     return r
36 def in_rs(M,v): M=np.array(M,int)%2; return rank(M)==rank(np.vstack([M
37     ,v]))
38 def swap(v): n=len(v)//2; return np.concatenate([v[n:],v[:n]])
39 print(f"{'code':16s}{'n':>3s}{'d':>3s}{'CSS':>7s}{'gen wt':>9s}{'H
40     transv':>10s}")
41 for name,G in CODES.items():
42     n=len(G[0])//2; css=all(is_pure(g) for g in G)
43     wt="/".join(str(w) for w in sorted({weight(g) for g in G}))
44     Hok=all(in_rs(np.array(G),swap(g)) for g in G)
45     print(f"{'name':16s}{'n':>3d}{'3':>3d}{'str(css):>7s}{'wt':>9s}{'str(Hok
46         ):>10s}")

```

出力:

code	n	d	CSS	gen wt	H transv
Shor [[9,1,3]]	9	3	True	2/6	False
Five [[5,1,3]]	5	3	False	4	False
Steane[[7,1,3]]	7	3	True	4	True

表が示すとおり、距離 3・論理 1 量子ビットは三符号で共通だが、Steane だけが二つの軸で際立つ。第一の軸は**自己双対性**—— Steane は $H^{\otimes 7}$ が生成子群を保つ（表の H transv = True）ため、アダマール・位相・CNOT という Clifford の生成元すべてが transversal にかかる（観察 1）。Shor は CSS だが非自己双対で H が transversal に届かず、五量子ビットは非 CSS で一式が届かない。第二の軸は**weight の低さ**—— Steane の生成子はすべて weight

4 で、症候群測定が触る量子ビットが少ない。これは故障の伝わる経路が短いことを意味し、しきい値を高くする（観察 3）。耐性を保って論理計算を積み上げる土台としては、「Clifford を丸ごと安く回せる（自己双対）」かつ「測定が軽くしきい値が高い（低 weight）」の両方を満たす Steane が、Clifford を一式 transversal にできない Shor とも、最小だが非 CSS で扱いにくい五量子ビットとも違う位置に立つ。

演習 10-2（スタビライザ状態が古典計算機で追える — Gottesman–Knill）

スタビライザ状態が古典計算機で効率よく追えること — Gottesman–Knill の定理 — を手元で体感する。まずアダマール 1 つと CNOT 1 つからなる Clifford 回路 ($|00\rangle$ から論理ベル状態に相当する状態を作る) を状態ベクトルで実行し、その状態を +1 で固定する生成子を総当りで読み取る。次に状態ベクトルを一切持たず、生成子だけを Clifford の共役規則（各 Pauli がどの Pauli に移るか）で更新する簡易タブローを実装し、両者が同じ生成子を与えることを確かめる。最後に T ゲートを混ぜ、簡易版が破れることを見る。

```

1 import numpy as np, itertools
2 I2=np.eye(2); X=np.array([[0,1],[1,0]],dtype=complex)
3 Y=np.array([[0,-1j],[1j,0]]); Z=np.array([[1,0],[0,-1]],dtype=complex)
4 Hm=np.array([[1,1],[1,-1]],dtype=complex)/np.sqrt(2)
5 CNOT=np.array([[1,0,0,0],[0,1,0,0],[0,0,0,1],[0,0,1,0]],dtype=complex)
6 # statevector side: H then CNOT on |00> -> Bell
7 psi=np.zeros(4,dtype=complex); psi[0]=1
8 psi=np.kron(Hm,I2)@psi; psi=CNOT@psi
9 P1={"I":I2,"X":X,"Y":Y,"Z":Z}
10 fixers=[a+b for a,b in itertools.product("IXYZ",repeat=2)
11         if (a,b)!=("I","I") and np.allclose(np.kron(P1[a],P1[b])@psi,
12         psi)]
13 print("statevector stabilizers (fix Bell):", sorted(fixers))
14 # tableau side: track generators by conjugation, no statevector
15 def Hc(g,q): g=g.copy(); g[q],g[2+q]=g[2+q],g[q]; return g
16 def Cc(g,c,t): g=g.copy(); g[t]^=g[c]; g[2+c]^=g[2+t]; return g
17 def ts(g):
18     s=""
19     for q in range(2):
20         x,z=g[q],g[2+q]; s+="Y" if (x and z) else "X" if x else "Z" if
21         z else "I"
22     return s
23 gens=[np.array([0,0,1,0]),np.array([0,0,0,1])] # Z0, Z1 (start |00>)
24 gens=[Hc(g,0) for g in gens]; gens=[Cc(g,0,1) for g in gens]
25 print("tableau stabilizers (gens only):", sorted(ts(g) for g in
26         gens))
27 # break: T maps X out of the Pauli group
28 T=np.array([[1,0],[0,np.exp(1j*np.pi/4)]]); TXT=T@X@T.conj().T
29 def prop(M):
30     for nm,P in {"I":I2,"X":X,"Y":Y,"Z":Z}.items():

```

```

28         i,j=np.argwhere(np.abs(P)>1e-9)[0]; c=M[i,j]/P[i,j]
29         if np.allclose(M,c*P): return nm
30     return None
31 print("T X T^dag is a single Pauli? ->", prop(TXT), "(None => tableau
breaks)")

```

出力:

```

statevector stabilizers (fix Bell): ['XX', 'ZZ']
tableau stabilizers (gens only):      ['XX', 'ZZ']
T X T^dag is a single Pauli? -> None (None => tableau breaks)

```

状態ベクトルから読んだ安定化生成子 $\{XX, ZZ\}$ と、生成子だけを共役規則で追った簡易タブローの結果 $\{XX, ZZ\}$ は一致する —— 2^n 次元のベクトルを一切持たずに、生成子 n 本を Pauli の移り先で書き換えるだけで Clifford 回路が追えた。これが Gottesman–Knill の中身であり、スタビライザ符号の症候群計算が古典的に軽い理由でもある。一方 T ゲートで X を共役すると $(X+Y)/\sqrt{2}$ になり、これは単一の Pauli ではない（出力の None）—— 生成子を「ある Pauli から別の Pauli へ」と書き換える規則が適用できず、簡易タブローはもう状態を追えない。Clifford の外に一步出た瞬間に、スタビライザによる効率的な記述が破れる。非 Clifford ゲートを耐性込みで供給する代償（魔法状態の蒸留など）が、万能性の議論で主題になる。

演習 10-3〔発展〕（五量子ビットの非 CSS 性と transversal H の不在）

五量子ビットが CSS でないこと、そして Clifford を一式 transversal に持たないことを調べる。生成子は $XZZXI$ とその巡回シフトの 4 本。(1) 各生成子が X を持つ桁と Z を持つ桁を併せ持ち、純 X 型・純 Z 型に分けられない（非 CSS）こと、(2) 各量子ビットに H を一様にかける $H^{\otimes 5}$ が生成子の集合を自分自身に戻さない（transversal H が論理ゲートにならない）ことを、生成子を H で共役して確かめる。Steane では $H^{\otimes 7}$ が集合を保った（第 9 章）のと対照する。

```

1  import numpy as np
2  FIVE=["XZZXI","IXZZX","XIXZZ","ZXIXZ"]
3  def symp(s):
4      n=len(s); x=np.zeros(n,int); z=np.zeros(n,int)
5      for i,c in enumerate(s):
6          if c in "XY": x[i]=1
7          if c in "ZY": z[i]=1
8      return np.concatenate([x,z])
9  def hasX(v): n=len(v)//2; return bool(v[:n].any())
10 def hasZ(v): n=len(v)//2; return bool(v[n:].any())
11 def rank(M):
12     M=(np.array(M,int)%2).copy(); r=0
13     for c in range(M.shape[1]):

```

```

14         p=next((i for i in range(r,M.shape[0]) if M[i,c]),None)
15         if p is None: continue
16         M[[r,p]]=M[[p,r]]
17         for i in range(M.shape[0]):
18             if i!=r and M[i,c]: M[i]=(M[i]+M[r])%2
19         r+=1
20     return r
21 def in_rs(M,v): M=np.array(M,int)%2; return rank(M)==rank(np.vstack([M
    ,v]))
22 def swap(v): n=len(v)//2; return np.concatenate([v[n:],v[:n]])
23 G=[symp(s) for s in FIVE]
24 print("each generator mixes X and Z (=> not CSS form):")
25 for s,g in zip(FIVE,G): print(f"    {s}: has X and Z = {hasX(g) and
    hasZ(g)}")
26 print("    all four mixed:", all(hasX(g) and hasZ(g) for g in G))
27 print("H^(x)5 preserves generator group?",
28       all(in_rs(np.array(G),swap(g)) for g in G), "(transversal H not
    logical)")
29 H=np.array([[0,0,0,1,1,1,1],[0,1,1,0,0,1,1],[1,0,1,0,1,0,1]])
30 S=[np.concatenate([r,np.zeros(7,int)]) for r in H]+[np.concatenate([np
    .zeros(7,int),r]) for r in H]
31 print("Steane H^(x)7 preserves group?",
32       all(in_rs(np.array(S),swap(g)) for g in S), "(transversal H IS
    logical)")

```

出力:

each generator mixes X and Z (=> not CSS form):

XZZXI: has X and Z = True

IXZZX: has X and Z = True

XIXZZ: has X and Z = True

ZXIXZ: has X and Z = True

all four mixed: True

H^(x)5 preserves generator group? False (transversal H not logical)

Steane H^(x)7 preserves group? True (transversal H IS logical)

四本の生成子はいずれも X を持つ桁と Z を持つ桁を併せ持ち、純 X 型・純 Z 型のどちらにも分類できない —— これが非 CSS の現れである (CSS 符号なら X 型と Z 型に分離した生成子の組を取れる)。 $H^{\otimes 5}$ は各桁で $X \leftrightarrow Z$ を入れ替えるが、入れ替えた生成子は元の生成子群に戻らない (出力 False) —— したがって transversal H は五量子ビットの論理アダマールにならない。Steane では $H^{\otimes 7}$ が群を保ち (出力 True)、自己双対性ゆえに transversal H がそのまま論理 H として働いた (第 9 章)。五量子ビットは距離 3 を最小の 5 量子ビットで達成する完全符号だが、その代償として CSS 構造と Clifford の transversal 性を手放している —— 符号の「小ささ」と「扱いやすさ」は別物だと、生成子の形が教えてくれる。